



UNIVERSITÉ MARIE & LOUIS PASTEUR  
DÉPARTEMENT INFORMATIQUE  
Master 2 en Ingénierie Système et Logiciel

---

EVOLUTION DU LANGAGE CHIPS : AMÉLIORATION DU  
COMPILATEUR ET INTÉGRATION DE COMPOSANTS DE LA  
THÉORIE DU CONTRÔLE

---

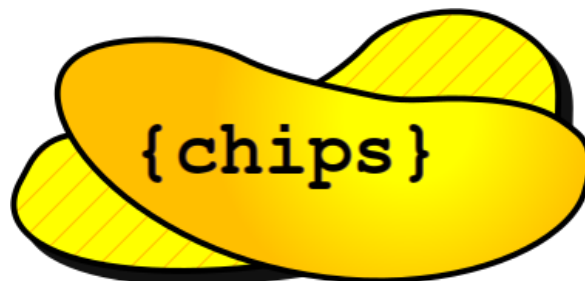
FOCHEUX VITAL

TUTRICES :

CERF SOPHIE

GALLONE ANNA

KOUCHNARENKO OLGA



---

RAPPORT DE PROJET DE MASTER 2 INGÉNIERIE SYSTÈME  
ET LOGICIEL - ANNÉE 2025/2026

## Remerciements

Je tiens à exprimer mes remerciements envers Madame Olga KOUCHNARENKO, ma maître de stage ainsi que tutrice universitaire, pour m'avoir proposé et encadré tout au long de ce stage.

Je remercie également Madame Anna GALLONE, pour tous les conseils qu'elle m'a apportés tout au long de ce stage par rapport au langage CHIPS, à certains concepts mathématiques et à la théorie du contrôle.

Mes remerciements vont également à Madame Sophie CERF, pour m'avoir guidé et conseillé sur des concepts de la théorie du contrôle.

Enfin, je souhaite exprimer ma reconnaissance envers l'ensemble de l'équipe du DISC, pour m'avoir accueilli et intégré dans l'équipe VESONTIO, ainsi qu'à toutes les personnes qui ont pu m'aider de près ou de loin dans la réalisation de ce stage.

# Table des matières

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                       | <b>6</b>  |
| <b>2</b> | <b>Présentation FEMTO-ST</b>                                              | <b>7</b>  |
| 2.1      | FEMTO-ST . . . . .                                                        | 7         |
| 2.2      | DISC . . . . .                                                            | 7         |
| 2.3      | Projet ADAPT . . . . .                                                    | 7         |
| <b>3</b> | <b>Littérature, contexte scientifique et objectifs du projet</b>          | <b>9</b>  |
| 3.1      | Contexte scientifique . . . . .                                           | 9         |
| 3.2      | Objectifs . . . . .                                                       | 11        |
| <b>4</b> | <b>Le parseur et la sérialisation XMI</b>                                 | <b>13</b> |
| 4.1      | Le parseur . . . . .                                                      | 13        |
| 4.1.1    | Migration vers ANTLR4 . . . . .                                           | 14        |
| 4.1.2    | La table des symboles pour le parseur . . . . .                           | 14        |
| 4.2      | Sérialisation vers XMI . . . . .                                          | 16        |
| 4.2.1    | Evolution du visiteur de génération du XMI . . . . .                      | 16        |
| 4.2.2    | La table des symboles pour le XMI . . . . .                               | 17        |
| <b>5</b> | <b>Implantation des outils de la théorie du contrôle</b>                  | <b>18</b> |
| 5.1      | Les filtres . . . . .                                                     | 18        |
| 5.1.1    | Le filtre passe-Bas (ordre 1) . . . . .                                   | 21        |
| 5.1.2    | Le filtre passe-Haut (ordre 1) . . . . .                                  | 25        |
| 5.1.3    | Le filtre Butterworth (ordre 2) . . . . .                                 | 27        |
| 5.1.4    | Le filtre médian . . . . .                                                | 30        |
| 5.1.5    | Evaluation des filtres sur des signaux d'instruments de musique . . . . . | 33        |
| 5.2      | L'implémentation des anti-windups . . . . .                               | 40        |
| 5.2.1    | L'anti-windup clamp . . . . .                                             | 40        |
| 5.2.2    | L'anti-windup back-calculation . . . . .                                  | 42        |
| 5.2.3    | Illustration par un cas pratique . . . . .                                | 43        |
| 5.2.4    | La configuration du système de contrôle . . . . .                         | 43        |
| 5.2.5    | Evaluation des performances . . . . .                                     | 45        |
| 5.3      | La chaîne d'outils . . . . .                                              | 51        |
| 5.3.1    | Le fonctionnement de la chaîne d'outils . . . . .                         | 51        |
| 5.3.2    | Les résultats de la chaîne d'outils . . . . .                             | 51        |
| 5.3.3    | Les métriques de la chaîne d'outils . . . . .                             | 57        |
| <b>6</b> | <b>Conclusion</b>                                                         | <b>58</b> |
| 6.1      | Bilan technique . . . . .                                                 | 58        |
| 6.2      | Les perspectives . . . . .                                                | 58        |
| 6.3      | Bilan personnel . . . . .                                                 | 59        |
| <b>A</b> | <b>Annexe</b>                                                             | <b>60</b> |
| <b>B</b> | <b>Glossaire</b>                                                          | <b>65</b> |
|          | <b>Références</b>                                                         | <b>66</b> |

## Table des figures

|    |                                                                                                                                            |    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Architecture de base avec les parties modifiées . . . . .                                                                                  | 12 |
| 2  | Architecture du parseur et du générateur XMI . . . . .                                                                                     | 14 |
| 3  | Diagramme de classes des filtres . . . . .                                                                                                 | 20 |
| 4  | Résultat du filtre passe-bas d'ordre 1 sur des fréquences avec du bruit . . . . .                                                          | 24 |
| 5  | Spectre fréquentiel du signal avant et après filtre passe-bas d'ordre 1 . . . . .                                                          | 24 |
| 6  | Résultat du filtre passe-haut d'ordre 1 sur des fréquences avec du bruit . . . . .                                                         | 26 |
| 7  | Spectre fréquentiel du signal avant et après filtre passe-haut d'ordre 1 . . . . .                                                         | 27 |
| 8  | Résultat du filtre passe-bas d'ordre 2 sur des fréquences avec du bruit . . . . .                                                          | 29 |
| 9  | Spectre fréquentiel du signal avant et après filtre passe-bas d'ordre 2 . . . . .                                                          | 30 |
| 10 | Résultat du filtre passe-haut d'ordre 2 sur des fréquences avec du bruit . . . . .                                                         | 31 |
| 11 | Spectre fréquentiel du signal avant et après filtre passe-haut d'ordre 2 . . . . .                                                         | 31 |
| 12 | Résultat du filtre médian sur des fréquences avec des pics de bruit, avec une fenêtre de 5 . . . . .                                       | 32 |
| 13 | Résultat du filtre médian sur des fréquences avec des pics de bruit, avec une fenêtre de 10 . . . . .                                      | 33 |
| 14 | Résultat du filtre médian sur des fréquences avec des pics de bruit, avec une fenêtre de 15 . . . . .                                      | 34 |
| 15 | Spectre fréquentiel de la piste audio de la basse . . . . .                                                                                | 35 |
| 16 | Densité spectrale de puissance de la piste audio de la basse . . . . .                                                                     | 36 |
| 17 | Spectre fréquentiel de la piste audio de la guitare . . . . .                                                                              | 36 |
| 18 | Densité spectrale de puissance de la piste audio de la guitare . . . . .                                                                   | 37 |
| 19 | Spectre fréquentiel de la piste audio de la flûte . . . . .                                                                                | 38 |
| 20 | Densité spectrale de puissance de la piste audio de la flûte . . . . .                                                                     | 38 |
| 21 | Principe de l'anti-windup sur l'intégral du PID . . . . .                                                                                  | 41 |
| 22 | Schéma bloc fonctionnel du système de contrôle du drone . . . . .                                                                          | 43 |
| 23 | Altitude du drone en avec différents anti-windups avec la retenue . . . . .                                                                | 48 |
| 24 | Altitude du drone en avec différents anti-windups sans la retenue . . . . .                                                                | 49 |
| 25 | Intégrale des différents anti-windups avec la retenue . . . . .                                                                            | 49 |
| 26 | Intégrale des différents anti-windups sans la retenue . . . . .                                                                            | 50 |
| 27 | Vitesse de rotation des hélices avec différents anti-windups avec retenue . . . . .                                                        | 50 |
| 28 | Vitesse de rotation des hélices avec différents anti-windups sans retenue . . . . .                                                        | 51 |
| 29 | Les différentes altitudes du drone avec différents anti-windups comparés à sans anti-windup en CHIPS . . . . .                             | 52 |
| 30 | Les différentes intégrales des anti-windups comparées à sans anti-windup en CHIPS . . . . .                                                | 53 |
| 31 | Les différentes vitesses de rotation des hélices avec différents anti-windups comparées à sans anti-windup en CHIPS . . . . .              | 53 |
| 32 | Les différentes altitudes du drone avec différents anti-windups comparés à l'anti-windup clamp en CHIPS . . . . .                          | 54 |
| 33 | Les différentes intégrales des anti-windups comparées à l'anti-windup clamp en CHIPS . . . . .                                             | 54 |
| 34 | Les différentes vitesses de rotation des hélices avec différents anti-windups comparés à l'anti-windup clamp en CHIPS . . . . .            | 55 |
| 35 | Les différentes altitudes du drone avec différents anti-windups comparés à l'anti-windup back-calculation en CHIPS . . . . .               | 55 |
| 36 | Les différentes intégrales des anti-windups comparées à l'anti-windup back-calculation en CHIPS . . . . .                                  | 56 |
| 37 | Les différentes vitesses de rotation des hélices avec différents anti-windups comparés à l'anti-windup back-calculation en CHIPS . . . . . | 56 |
| 38 | Densité spectrale de puissance avant et après filtre passe-bas d'ordre 1 . . . . .                                                         | 60 |

|    |                                                                                     |    |
|----|-------------------------------------------------------------------------------------|----|
| 39 | Densité spectrale de puissance avant et après filtre passe-haut d'ordre 1 . . . . . | 61 |
| 40 | Densité spectrale de puissance avant et après filtre passe-bas d'ordre 2 . . . . .  | 61 |
| 41 | Densité spectrale de puissance avant et après filtre passe-haut d'ordre 2 . . . . . | 62 |

## Liste des tableaux

|   |                                                                                                                            |    |
|---|----------------------------------------------------------------------------------------------------------------------------|----|
| 1 | Comparaison des résultats avec et sans retenue sur l'altitude maximale atteinte par le drone. . . . .                      | 46 |
| 2 | Comparaison des résultats avec et sans retenue sur le dépassement maximal de l'altitude atteinte par le drone. . . . .     | 47 |
| 3 | Comparaison des résultats avec et sans retenue sur le temps de stabilisation du drone. . . . .                             | 47 |
| 4 | Comparaison des résultats avec et sans retenue sur l'intégrale maximale atteinte par le contrôleur du drone. . . . .       | 47 |
| 5 | Comparaison des résultats avec et sans retenue sur la commande maximale générée par le contrôleur du drone. . . . .        | 47 |
| 6 | Comparaison des résultats avec et sans retenue sur la commande minimale locale générée par le contrôleur du drone. . . . . | 48 |
| 7 | Comparaison des résultats avec et sans retenue sur la commande moyenne générée par le contrôleur du drone. . . . .         | 48 |

# 1 Introduction

Le développement de systèmes logiciels complexes nécessite aujourd'hui des environnements capables d'accompagner les différentes étapes de leur conception, de leur validation et de leur évolution. Dans le domaine des systèmes à composants, ces environnements doivent non seulement permettre de modéliser des architectures de plus en plus élaborées, mais également de proposer des outils capables d'évoluer avec les besoins des utilisateurs et des projets de recherche.

Ce rapport s'inscrit dans le cadre du stage de Master 2 Ingénierie Système et Logiciel. Réalisé au sein de l'équipe VESONTIO de l'institut FEMTO-ST, il contribue au développement du langage CHIPS et de son environnement logiciel. Les travaux effectués durant ce projet portent sur l'évolution de plusieurs composants de la chaîne de compilation ainsi que sur l'enrichissement des fonctionnalités offertes par le langage.

Les objectifs poursuivis durant ce projet sont multiples. Ils consistent d'une part à améliorer certains éléments de l'infrastructure logicielle afin de faciliter l'évolution du langage, et d'autre part à intégrer de nouveaux composants réutilisables destinés à étendre les possibilités offertes aux utilisateurs. Enfin, une étude expérimentale est menée afin d'évaluer les performances des différentes solutions développées et leur impact sur la chaîne de compilation.

Ce rapport est organisé en six sections.

La section 2 présente la structure d'accueil dans laquelle s'inscrit ce stage.

La section 3 expose le contexte scientifique nécessaire à la compréhension des travaux réalisés.

La section 4 décrit les évolutions apportées au compilateur du langage CHIPS.

La section 5 présente les nouveaux composants développés ainsi que leur évaluation expérimentale.

Enfin, la section 6 conclut ce rapport en présentant un bilan des travaux réalisés et plusieurs perspectives d'évolution.

## 2 Présentation FEMTO-ST

### 2.1 FEMTO-ST

L'institut FEMTO-ST (Franche-Comté Electronique Mécanique Thermique et Optique - Sciences et Technologies) est une unité mixte de recherche associée au CNRS (UMR 6174) et rattachée simultanément à :

- l'université Marie-et-Louis-Pasteur (UMLP)
- l'Ecole nationale supérieure de mécanique et des microtechniques de Besançon (Supmicrotech)
- l'université de technologie de Belfort-Montbéliard (UTBM)

Cet institut a été créé le 1<sup>er</sup> janvier 2004 par la fusion de cinq laboratoires de Franche-Comté actifs dans les domaines de la mécanique, de l'optique et des télécommunications, de l'électronique, du temps-fréquence, de l'énergétique et de la fluïdique. Au 1<sup>er</sup> janvier 2012, il incorpore le laboratoire d'informatique de l'UMLP, qui devient le 7<sup>e</sup> département de l'institut.

FEMTO-ST compte aujourd'hui plus de 700 membres et constitue l'un des plus gros instituts de recherche dans les sciences de l'ingénieur en France. Il est présent en Franche-Comté sur trois sites géographiques, à Besançon, Belfort et Montbéliard.

### 2.2 DISC

Le DISC (Département d'Informatique et Systèmes Complexes) est le département de FEMTO-ST qui traite des sciences informatiques. L'objectif principal des recherches menées étant de modéliser, simuler, développer et valider les systèmes complexes. Le département, qui compte environ 100 personnes, est structuré en 4 équipes de recherche :

- Algorithmique Numérique Distribuée (AND)
- Design, Evaluation and Optimisation of Distributed and Intelligent Systems (DEODIS)
- Optimisation, Mobility, NetworkIng (OMNI)
- Vérification et validation de logiciels et de systèmes embarqués (VESONTIO)

Ce stage se déroule au sein de l'équipe VESONTIO dont l'objectif est porté sur la qualité logicielle et le développement de travaux pour améliorer ou analyser la fiabilité, la sécurité ou la performance des systèmes.

### 2.3 Projet ADAPT

Le projet ANR-23-CE25-0004, intitulé ADAPT (Adaptation dynamique de systèmes à composants hiérarchiques), a pour objectif principal de développer un cadre formel et des outils permettant de concevoir des systèmes cyber-physiques adaptatifs de manière correcte par construction. Il s'intéresse en particulier aux systèmes composés d'un grand nombre de composants modulaires, comme les robots modulaires Blinky Blocks, capables de se reconfigurer dynamiquement pour accomplir une tâche commune.

Les objectifs scientifiques du projet sont les suivants :

1. Étendre les modèles à composants hiérarchiques afin de permettre la description de systèmes complexes composés de nombreux modules. Cette modélisation hiérarchique doit faciliter la gestion de la complexité en représentant les composants à différents niveaux d'abstraction tout en intégrant les contraintes fonctionnelles et de ressources.



2. Intégrer des mécanismes de contrôle adaptatif directement dans ces modèles. L'objectif est de permettre aux systèmes de modifier automatiquement leur configuration en fonction des conditions d'exécution, qu'elles soient :
  - cyber, comme l'état du système, les interactions entre composants ou la topologie du réseau
  - physiques, comme la consommation d'énergie, le bruit des capteurs ou les perturbations de l'environnement.
3. Implémenter ces mécanismes d'adaptation dans une infrastructure de modélisation fondée sur les systèmes à composants, afin de fournir une chaîne complète allant de la modélisation jusqu'à l'exécution des stratégies d'adaptation.
4. Valider les mécanismes proposés par simulation avant leur déploiement sur des plateformes robotiques réelles. Les expérimentations sont réalisées à l'aide des outils DR-BIP et VisibleSim, puis appliquées à des robots modulaires tels que les Blinky Blocks.

Au sein de ce projet, il y a le développement d'un langage de programmation dédié à la modélisation formelle de systèmes distribués complexes, appelé **CHIPS** (Control of Hierarchical Interconnected Programmable Systems). Ce langage est basé sur le langage BIP (Behavior, Interaction, Priority) et permet de décrire des systèmes hiérarchiques composés de composants modulaires.

Il veut permettre d'intégrer des mécanismes de contrôle adaptatif directement dans les modèles de systèmes à composants hiérarchiques, afin de permettre aux systèmes de modifier automatiquement leur configuration en fonction des conditions d'exécution.

### 3 Littérature, contexte scientifique et objectifs du projet

Ce section présente le contexte scientifique dans lequel s’inscrivent les travaux réalisés durant ce projet.

Il introduit les principaux concepts nécessaires à la compréhension des développements présentés dans les sections suivantes. Nous présenterons tout d’abord les principes de l’ingénierie dirigée par les modèles ainsi que le fonctionnement général d’une chaîne de compilation. Nous introduirons ensuite les notions de théorie du contrôle mobilisées durant ce projet avant de présenter les principales problématiques auxquelles répondent les développements réalisés.

#### 3.1 Contexte scientifique

Les systèmes embarqués et les systèmes cyber-physiques occupent aujourd’hui une place importante dans de nombreux domaines industriels tels que la robotique, les transports, l’aéronautique ou encore les systèmes autonomes. Ces systèmes combinent des composants logiciels, des dispositifs matériels, des capteurs et des actionneurs devant interagir de manière fiable dans des environnements souvent complexes et évolutifs. Leur conception représente un défi important, tant par le nombre de composants impliqués que par les nombreuses interactions existant entre eux.

Afin de maîtriser cette complexité, l’ingénierie dirigée par les modèles (Model-Driven Engineering, MDE) propose de placer les modèles au coeur du processus de développement. Plutôt que de développer directement une application sous forme de code source, les systèmes sont d’abord décrits à un niveau d’abstraction élevé au moyen de modèles. Ces derniers peuvent ensuite être analysés, transformés et progressivement raffinés jusqu’à produire du code exécutable. Cette approche améliore la maintenabilité des systèmes, facilite leur évolution et permet d’automatiser une partie importante du développement logiciel grâce aux transformations de modèles.

Dans cette approche, les langages spécifiques à un domaine (Domain-Specific Languages, DSL) occupent une place centrale. Contrairement aux langages généralistes, ils offrent des abstractions directement adaptées à un domaine d’application particulier, permettant aux concepteurs d’exprimer plus naturellement les concepts manipulés tout en réduisant les risques d’erreurs. Le langage CHIPS s’inscrit dans cette philosophie. Développé dans le cadre du projet ANR ADAPT, il est destiné à la modélisation de systèmes distribués hiérarchiques et sa compilation repose sur les principes du langage BIP (Behavior, Interaction, Priority), largement utilisé pour la modélisation formelle de systèmes à composants hétérogènes [2]. CHIPS permet ainsi de décrire des systèmes complexes tout en conservant une représentation exploitable par des outils de simulation, de vérification formelle et de génération de code.

Une chaîne de compilation destinée à un tel langage repose sur plusieurs étapes complémentaires. Dans un premier temps, le programme est soumis à une analyse lexicale, qui consiste à transformer la suite de caractères du fichier source en une suite de termes. Cette étape est suivie d’une analyse syntaxique, dont l’objectif est de vérifier que ces lexèmes respectent la grammaire du langage et de construire un arbre syntaxique abstrait (AST). L’AST constitue une représentation structurée du programme indépendante de sa syntaxe textuelle.

Une fois cet arbre obtenu, une analyse sémantique est réalisée. Celle-ci dépasse largement la simple vérification grammaticale puisqu’elle consiste notamment à contrôler le typage, la cohérence des déclarations, la visibilité des identifiants ou encore les règles de portée des variables. Ces vérifications nécessitent généralement l’utilisation d’une table des symboles, structure

fondamentale dans la conception des compilateurs, permettant d'associer à chaque identifiant l'ensemble des informations nécessaires à son interprétation.

Dans le cadre de CHIPS, cette analyse sémantique est suivie d'une phase de génération de modèles XMI conforme au méta-modèle Ecore utilisé par l'Eclipse Modeling Framework (EMF). Cette représentation intermédiaire permet ensuite aux différents outils de la chaîne de compilation de manipuler le programme sous une forme standardisée avant sa transformation vers les représentations utilisées par le compilateur BIP. L'organisation générale de cette chaîne est présentée sur la figure 1.

Les premières versions du compilateur CHIPS reposaient sur les outils Flex, Bison et Yacc, particulièrement répandus dans le développement de compilateurs. Cette architecture a permis de démontrer la faisabilité du projet et de produire une partie de la chaîne de compilation fonctionnelle, qui est celle du parseur. Cependant, l'évolution progressive du langage a rapidement fait apparaître plusieurs limitations. L'introduction des templates C++, utilisées pour représenter les classes lors de la création de l'AST, rend notamment cette création plus complexe car les templates ne sont pas directement supportés par Bison. Afin de faciliter l'évolution du langage, une migration vers ANTLR4 a été envisagée. Cet outil permet de produire une grammaire plus lisible, plus modulaire et généralement plus simple à maintenir que Bison. De plus, ANTLR4 permet l'utilisation de templates C++ dans les actions de la grammaire, ce qui simplifie la création de l'AST.

Au-delà des évolutions du compilateur, CHIPS vise également à intégrer directement dans le langage des mécanismes issus de la théorie du contrôle. Les systèmes cyber-physiques reposent en effet sur des boucles d'asservissement utilisant les mesures fournies par des capteurs afin de commander des actionneurs. La qualité de ces mesures conditionne directement les performances du système de commande.

Parmi les régulateurs existants, le contrôleur PID (Proportional-Integral-Derivative) demeure aujourd'hui le plus largement utilisé dans l'industrie. De nombreuses études estiment que plus de 90 % des boucles de régulation industrielles utilisent encore un régulateur de type PI ou PID en raison de leur simplicité de mise en oeuvre, de leur faible coût de calcul et de leur robustesse pour un très grand nombre de procédés industriels [1]. Son principe consiste à combiner trois actions complémentaires : une action proportionnelle corrigeant instantanément l'erreur, une action intégrale supprimant les erreurs permanentes et une action dérivée permettant d'anticiper les variations du système. Malgré l'apparition de méthodes de commande plus avancées telles que la commande optimale ou la commande prédictive, les PID restent aujourd'hui la solution privilégiée dans de nombreux systèmes industriels grâce à leur excellent compromis entre simplicité et performances.

L'utilisation d'un PID soulève néanmoins deux difficultés majeures rencontrées dans la plupart des applications réelles.

La première concerne les mesures issues des capteurs. Les capteurs sont soumis à différentes perturbations : bruit électronique, vibrations mécaniques, erreurs de mesure, perturbations environnementales ou encore quantification numérique. Si ces perturbations sont directement utilisées par le contrôleur, elles peuvent provoquer des oscillations de la commande, une diminution de la stabilité ou une dégradation des performances du système. Il est donc nécessaire d'effectuer un prétraitement des signaux avant leur exploitation.

Les filtres numériques constituent l'une des solutions les plus utilisées pour répondre à cette

problématique. Les filtres passe-bas permettent d'atténuer les composantes de haute fréquence tout en conservant les variations lentes correspondant généralement au signal utile. À l'inverse, les filtres passe-haut suppriment dans un signal les composantes de basse fréquence. Parmi les nombreuses familles de filtres existantes, les filtres de Butterworth, proposés par Stephen Butterworth en 1930, sont particulièrement utilisés en automatique car ils présentent une réponse fréquentielle maximalement plate dans la bande passante, c'est-à-dire sans ondulations, tout en offrant une pente d'atténuation plus importante que les filtres du premier ordre [3]. Leur conception ainsi que leurs propriétés sont largement décrites dans les ouvrages de référence consacrés au traitement numérique du signal [4, 5]. Ces caractéristiques en font un excellent compromis entre coût de calcul, stabilité numérique et efficacité du filtrage.

La seconde difficulté apparaît lorsque la commande calculée par le PID dépasse les capacités physiques de l'actionneur. Dans cette situation, l'actionneur entre en saturation tandis que le terme intégral du PID continue d'accumuler l'erreur. Ce phénomène, connu sous le nom d'intégral windup, provoque généralement une augmentation importante du dépassement, un ralentissement du temps de réponse ainsi qu'une perte de stabilité transitoire. Ce problème est particulièrement fréquent dans les systèmes réels où les actionneurs possèdent nécessairement des limites physiques.

Afin de limiter ces effets, de nombreuses stratégies d'anti-windup ont été proposées dans la littérature. Les deux approches les plus couramment utilisées sont le clamping, qui consiste à suspendre ou limiter l'intégration lorsque la commande est saturée, et le back-calculation, qui réinjecte l'erreur de saturation dans l'intégrateur afin d'accélérer son retour vers une valeur cohérente. Ces méthodes sont aujourd'hui largement employées dans les applications industrielles car elles permettent d'améliorer significativement le comportement transitoire des régulateurs PID tout en conservant une mise en oeuvre relativement simple. Une synthèse détaillée de ces approches est proposée par Tarbouriech et Turner, qui présentent également les principales problématiques de recherche encore ouvertes dans ce domaine [6].

## 3.2 Objectifs

Les travaux réalisés durant ce stage s'inscrivent dans cette double problématique. D'une part, ils visent à faire évoluer l'infrastructure du compilateur CHIPS afin d'améliorer sa maintenabilité, son extensibilité et sa capacité à prendre en charge les nouvelles évolutions du langage. D'autre part, ils enrichissent la bibliothèque de composants du langage en intégrant plusieurs outils issus de la théorie du contrôle directement réutilisables dans les modèles CHIPS.

Plus précisément, les objectifs de ce stage sont les suivants :

- migrer le parseur historique vers ANTLR4 afin d'obtenir une architecture plus robuste et plus facilement extensible
- adapter le générateur XMI au nouvel arbre syntaxique produit par ANTLR4
- développer une bibliothèque de filtres numériques comprenant des filtres passe-bas, passe-haut, Butterworth d'ordre 2 ainsi qu'un filtre médian
- implémenter plusieurs mécanismes d'anti-windup destinés aux contrôleurs PID, notamment les approches Clamp et Back-calculation
- intégrer l'ensemble de ces composants à la chaîne de compilation CHIPS

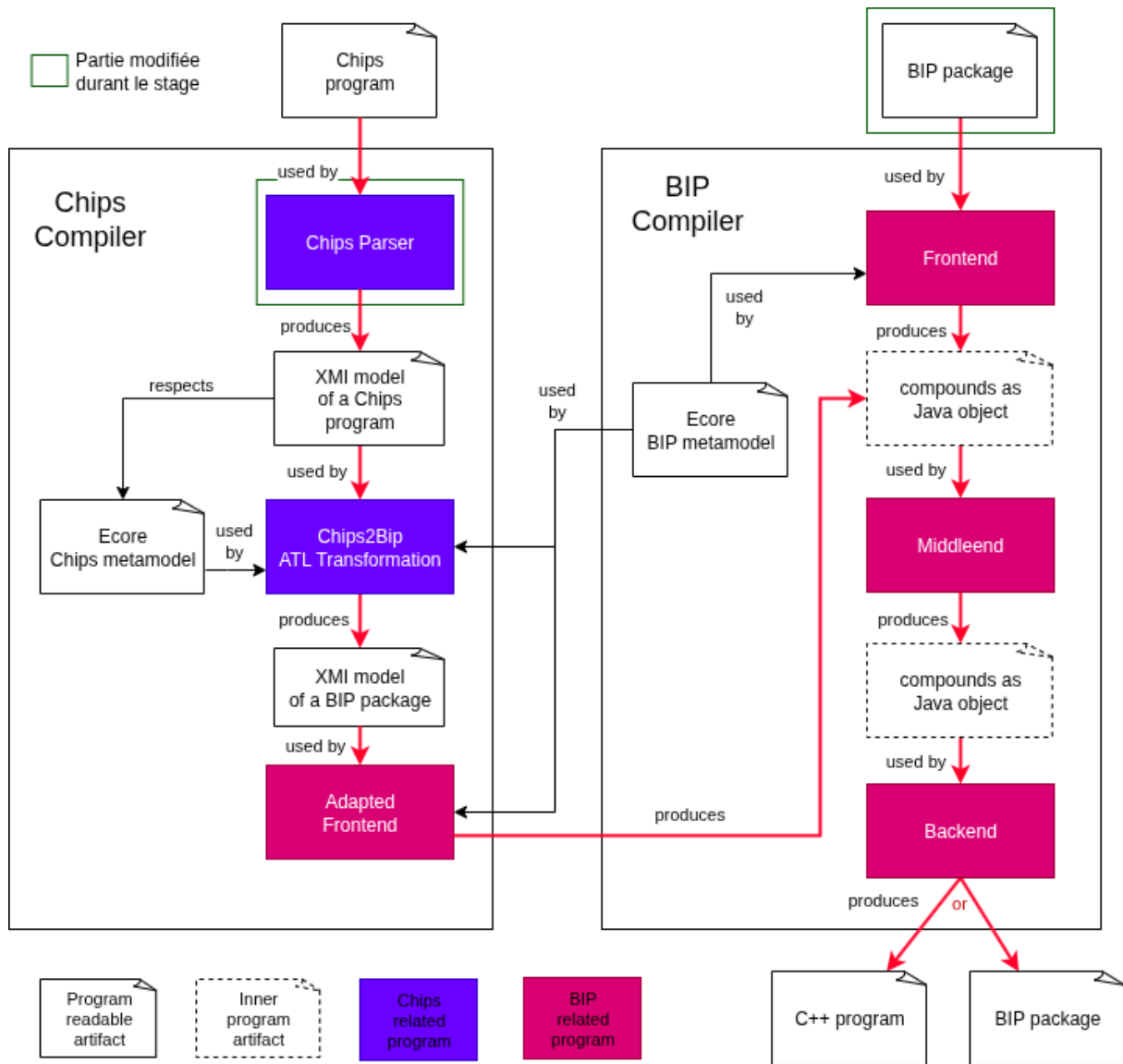


Figure 1: Architecture de base avec les parties modifiées

- évaluer expérimentalement les performances des différentes solutions proposées au moyen d'analyses temporelles, fréquentielles et de mesures de performances de la chaîne de compilation.

L'ensemble de ces travaux contribue ainsi à renforcer la maturité du langage CHIPS aussi bien du point de vue de son infrastructure logicielle que des fonctionnalités pour la modélisation de systèmes cyber-physiques adaptatifs.

## 4 Le parseur et la sérialisation XMI

L'un des principaux objectifs de ce projet est de permettre la transformation d'un programme écrit en CHIPS vers une représentation XMI conforme au méta-modèle Ecore du langage. Cette étape constitue un élément essentiel de la chaîne de modélisation, puisqu'elle permet de convertir une description textuelle en un modèle structuré pouvant être manipulé par différents outils reposant sur les technologies de l'Eclipse Modeling Framework (EMF). La difficulté ne réside pas uniquement dans la lecture du langage mais également dans l'interprétation correcte de sa structure afin de reconstruire fidèlement les relations définies par le méta-modèle.

### Note terminologique

Un méta-modèle est, par analogie, la "grammaire" d'un modèle. Dans le projet de CHIPS, il prend la forme de diagrammes de classes Ecore qui définissent de manière stricte les concepts du langage CHIPS (entités, attributs, relations). Le fichier chips1.1.ecore est donc le schéma directeur qui dicte comment un fichier XMI doit être structuré pour être valide.

Pour atteindre cet objectif, plusieurs évolutions ont été apportées à l'infrastructure existante. Le parseur historique, développé avec Yacc/Bison et Flex, ne permettait pas de prendre en charge certaines constructions modernes du langage, notamment les templates du C++, qui sont utilisés pour définir les classes génériques lors de la compilation. Une réécriture basée sur ANTLR4 a donc été entreprise afin de disposer d'une représentation syntaxique plus robuste et mieux adaptée à l'interprétation du langage. Cette nouvelle architecture facilite également le parcours de l'arbre syntaxique abstrait, étape indispensable à la construction du modèle XML.

Cette partie présente ainsi les différentes briques mises en œuvre pour réaliser cette transformation. Nous commencerons par décrire la migration vers ANTLR4 ainsi que la conception de la table des symboles utilisée lors de l'analyse du programme. Nous expliquerons ensuite comment le parcours de l'arbre syntaxique, réalisé au moyen du patron de conception visiteur, permet de construire progressivement le modèle correspondant. Enfin, nous détaillerons les adaptations apportées à la table des symboles afin de résoudre correctement les références entre les différents éléments du méta-modèle et de produire un fichier XMI valide.

Il y a deux tables des symboles distinctes à gérer (cf. figure 2), celle du parseur et celle du générateur XMI. Etant donné que le méta-modèle, pour être respecté, implique que le parseur fasse bien plus que parser. Il doit surtout faire le contrôle de type car le langage est fortement typé, mais aussi des vérifications des portées des variables dans le programme CHIPS.

### 4.1 Le parseur

Les premiers travaux consacrés au développement du langage CHIPS ont conduit à la réalisation d'un premier interpréteur reposant sur les outils Yacc/Bison et Flex. Cette architecture assurait l'analyse lexicale et syntaxique des programmes CHIPS afin de produire un arbre syntaxique abstrait représentant leur structure. Cet AST constituait ensuite le point d'entrée des différents traitements réalisés par l'interpréteur, notamment la génération d'un modèle XMI conforme au méta-modèle du langage.

Cette première implémentation a permis de démontrer la faisabilité de la chaîne d'interprétation du langage et de valider les principaux choix architecturaux retenus pour le projet. Elle constitue aujourd'hui encore la base sur laquelle reposent les développements réalisés pendant le

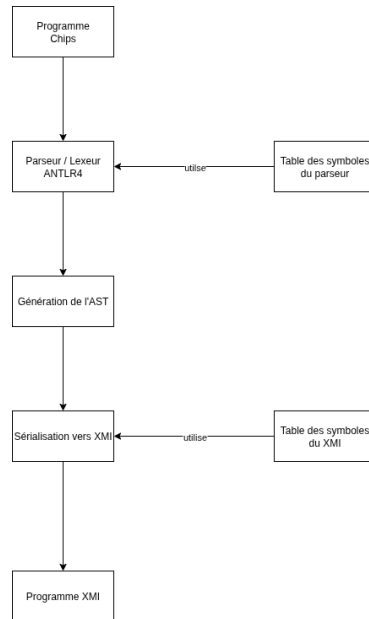


Figure 2: Architecture du parseur et du générateur XMI

stage.

Toutefois, l'évolution du langage CHIPS a progressivement fait apparaître de nouvelles exigences, notamment avec l'introduction des templates C++. Ces nouvelles constructions syntaxiques ont conduit à faire évoluer l'architecture de l'analyseur afin de conserver une grammaire claire et facilement extensible. Les sections suivantes présentent les principaux travaux réalisés dans cette optique.

#### 4.1.1 Migration vers ANTLR4

L'analyseur développé lors des travaux antérieurs reposait sur une grammaire écrite pour Yacc/Bison et Flex, permettant de reconnaître les différentes constructions du langage CHIPS et de construire l'arbre syntaxique abstrait correspondant. Cette architecture n'assurait pas fidèlement la compatibilité avec l'approche dirigée par les modèles, par conséquent pour augmenter cette fidélité de l'AST du parseur vis-à-vis du méta-modèle.

Au cours du stage, l'apparition des templates C++, permettant de définir des classes génériques passant de 297 à 97 classes C++, a introduit une nouvelle problématique. Ces constructions possèdent une syntaxe fortement imbriquée qui rend leur prise en charge particulièrement complexe dans la grammaire Bison existante. Afin de permettre leur intégration tout en conservant les fonctionnalités déjà développées, le choix a été fait de migrer vers ANTLR4, dont les mécanismes d'analyse offrent une plus grande souplesse pour décrire ce type de structures syntaxiques.

Cette migration constitue la principale évolution de la chaîne d'analyse réalisée pendant ce stage. Elle a nécessité l'adaptation des différents traitements reposant sur l'AST généré par l'analyseur.

#### 4.1.2 La table des symboles pour le parseur

Il a fallu implémenter une table des symboles assez robuste pour qu'elle permette de ne pas instancier deux classes différentes qui auraient le même nom dans un programme CHIPS. La classe

SymbolTable sert de registre central des noms visibles pendant l'analyse du programme. Son rôle est de mémoriser ce qui a été déclaré, avec son type logique, pour que le reste du parseur puisse retrouver rapidement une variable, une fonction, un channel, un objet ou un bloc sans ambiguïté.

## Structure interne

Elle repose sur plusieurs niveaux de stockage.

- **scopes** contient une pile de scopes, chaque scope étant une table associant un nom à un symbole.
- chaque **Symbol** stocke trois choses : la valeur réelle dans un **std:any**, le nom, et la catégorie du symbole via **SymbolKind**.
- en plus de ça, il y a deux tables séparées pour les sorties de fonctions et les paramètres de fonctions.
- enfin, **declared\_block\_system** garde le type associé à certains blocs déclarés.

L'idée générale est donc : un stockage principal pour les symboles ordinaires, et des structures annexes pour les cas particuliers liés aux fonctions et aux blocs.

### Points important

CHIPS est un langage qui définit des modèles sous la forme de blocs fonctionnels pour représenter un processus, en les connectant entre eux, on permet au modèle de réaliser une suite d'opérations.

## Méthodes principales

Les méthodes les plus importantes sont assez simples à lire.

- **enterScope()** et **exitScope()** ouvrent et ferment un scope.
- **declare...()** enregistre un symbole d'un certain genre : variable, variable contextuelle, channel, fonction, objet, bloc...
- **lookup...()** recherche un symbole dans les scopes, en partant du plus interne vers le plus externe.
- **declareFunctionOutput()** et **declareFunctionParameter()** gèrent les métadonnées des fonctions.
- **declareBlock()** et **getTypeOfDeclaredBlock()** servent à mémoriser puis relire le type d'un bloc.
- **dump()** affiche un état lisible de la table pour le débogage.

## Exemple d'utilisation

En pratique, l'utilisation de ces fonctions se réalise dans cet ordre :



```

1  auto& table = chips::SymbolTable::getInstance();
2  table.enterScope();
3  table.declareVariable("x", someValue);
4  // dom somethings
5  if(table.lookupVariable("x").has_value()){
6      // do somethings
7  }else{
8      // handle error
9  }
10 table.exitScope();

```

Listing 1: Exemple d'utilisation de SymbolTable

Ici, on entre dans un scope, on va déclarer une variable ayant pour identifiant "x". On va effectuer un certain nombre d'opérations, puis on va chercher à retrouver la variable "x" dans la table des symboles. Si elle est trouvée, on peut l'utiliser et effectuer des opérations dessus. Sinon, on gère l'erreur de manière appropriée. Enfin, on quitte le scope, ce qui supprime toutes les variables déclarées dans ce scope de la table des symboles.

## 4.2 Sérialisation vers XMI

Les travaux antérieurs ont également permis de développer une première chaîne de transformation capable de convertir un programme CHIPS en un modèle XMI conforme au méta-modèle Ecore. Cette étape constitue le coeur de l'interprétation du langage puisqu'elle permet de transformer une représentation textuelle en un modèle exploitable par les différents outils de l'ingénierie dirigée par les modèles. Cette première architecture reposait sur un parcours de l'arbre syntaxique abstrait afin de générer progressivement les différents éléments du document XMI.

Cette première implémentation a permis de démontrer la faisabilité de la sérialisation du langage et de mettre en place les principaux mécanismes nécessaires à la génération des modèles. Elle introduisait notamment un visiteur chargé du parcours de l'AST ainsi qu'une première table des symboles permettant de résoudre les références présentes dans le modèle généré.

L'évolution de l'interpréteur réalisée au cours du stage a toutefois nécessité de faire évoluer cette architecture (cf. figure 2) afin de l'adapter au nouvel AST produit par ANTLR4 tout en conservant les principes de fonctionnement établis par les travaux antérieurs.

### 4.2.1 Evolution du visiteur de génération du XMI

Les premiers travaux de sérialisation reposaient sur un visiteur chargé de parcourir récursivement l'arbre syntaxique abstrait afin de construire progressivement le modèle XMI correspondant. Chaque méthode de visite avait pour rôle de traduire un type de noeud particulier en son équivalent dans le méta-modèle Ecore tout en assurant la cohérence de la structure produite.

Le principe général de cette architecture a été conservé au cours du stage. En revanche, la migration vers ANTLR4 ainsi que l'évolution du langage CHIPS ont nécessité de faire évoluer plusieurs mécanismes internes du visiteur afin de prendre en compte la nouvelle organisation de l'AST et les nouvelles constructions syntaxiques introduites.

Les développements réalisés ont principalement porté sur la restructuration du visiteur, la factorisation de traits communs et l'amélioration de la génération des différents éléments du modèle XMI.

#### 4.2.2 La table des symboles pour le XMI

La génération d'un modèle XMI nécessite de retrouver précisément les différents éléments déclarés dans un programme afin de construire correctement les références imposées par le méta-modèle Ecore. Les travaux antérieurs répondaient à cette problématique au moyen d'une première table des symboles associant les identifiants rencontrés lors du parcours de l'AST aux chemins XMI des éléments correspondants.

Cette première architecture répondait aux besoins de la chaîne de transformation initiale mais devait évoluer afin d'accompagner les modifications apportées au nouvel AST ainsi que les nouvelles fonctionnalités du langage. Les mécanismes de résolution des références ont notamment été généralisés afin de prendre en compte davantage de catégories de symboles et de centraliser les informations nécessaires à la génération du modèle.

Les travaux réalisés au cours du stage ont ainsi conduit à une refonte de cette structure de données afin d'en faire un composant central de la sérialisation XMI. Cette évolution améliore la robustesse du processus de génération, facilite la résolution des références entre les différents éléments du modèle et prépare l'infrastructure aux futures évolutions du langage CHIPS.

## 5 Implantation des outils de la théorie du contrôle

Les travaux réalisés se sont concentrés sur deux problématiques majeures rencontrées dans les systèmes asservis. La première concerne le traitement des mesures provenant des capteurs. En pratique, ces mesures sont rarement parfaites : elles peuvent être affectées par du bruit, des perturbations ou des variations rapides susceptibles de dégrader le fonctionnement d'un contrôleur. La seconde problématique est liée à la saturation des actionneurs. Lorsqu'un contrôleur PID demande une commande supérieure aux capacités physiques du système, un phénomène d'accumulation de l'action intégrale, appelé intégral windup, peut apparaître et entraîner une dégradation importante des performances du système.

Afin de répondre à ces deux problématiques, plusieurs composantes ont été développées et intégrées dans une architecture commune afin de favoriser leur réutilisation et leur extensibilité. L'objectif n'était pas uniquement de proposer une implémentation fonctionnelle, mais également de comparer les différentes approches afin d'identifier les solutions les plus adaptées selon les contraintes d'utilisation. Une attention particulière a ainsi été portée à la simplicité d'intégration, au coût de calcul, à la stabilité numérique ainsi qu'aux performances obtenues sur des signaux représentatifs.

La section 5.1 est consacrée aux filtres numériques. Après avoir présenté l'architecture mise en place pour factoriser leur implémentation, nous étudierons successivement les filtres passe-bas, passe-haut, Butterworth d'ordre 2 et médian. Pour chacun d'entre eux, nous rappellerons les fondements théoriques, les équations utilisées lors de leur discrétisation ainsi que les choix d'implémentation retenus. Leurs performances seront ensuite évaluées au moyen d'analyses temporelles et fréquentielles, notamment à l'aide de la transformée de Fourier rapide (FFT) et de la densité spectrale de puissance (PSD), avant d'être comparées afin de mettre en évidence leurs avantages, leurs limites et leurs domaines d'application respectifs.

La section 5.2 est consacrée aux mécanismes d'anti-windup destinés à limiter les effets de la saturation dans les contrôleurs PID. Après une représentation des méthodes Clamp et Back-calculation, leur fonctionnement sera illustré sur un cas pratique représentatif mettant en évidence le phénomène de windup. Les différentes stratégies seront ensuite évaluées expérimentalement afin de comparer leur influence sur les performances du contrôleur, notamment en termes de stabilité, de temps de réponse, de dépassement et de vitesse de retour à un fonctionnement nominal. Cette étude permettra d'identifier les avantages et les limites de chacune des approches proposées, ainsi que les contextes dans lesquels elles sont les plus pertinentes.

### 5.1 Les filtres

Dans la théorie du contrôle, les filtres sont utilisés pour améliorer la qualité des mesures issues des capteurs avant qu'elles ne soient exploitées par le contrôleur. Les mesures réelles sont en effet souvent perturbées par du bruit, des parasites ou des variations rapides qui peuvent dégrader les performances d'un système asservi. Un filtrage approprié permet de réduire ces perturbations tout en conservant les informations utiles au contrôle.

L'étude des filtres repose sur deux outils fondamentaux de l'automatique : le domaine de Laplace et la fonction de transfert. Ces outils permettent de décrire mathématiquement le comportement dynamique d'un système de manière beaucoup plus simple que les équations différentielles utilisées dans le domaine temporel.

## Domaine de Laplace

Dans le domaine temporel, le comportement d'un système est généralement décrit par une équation différentielle. Par exemple, un filtre passe-bas du premier ordre peut être représenté par une équation faisant intervenir la dérivée de la sortie par rapport au temps.

Manipuler directement ces équations devient rapidement complexe lorsque les systèmes possèdent plusieurs états ou plusieurs dérivées. Afin de simplifier leur étude, on utilise la transformée de Laplace, qui convertit les équations différentielles en équations algébriques.

La transformée de Laplace d'un signal  $x(t)$  est définie par :

$$X(s) = \mathcal{L}\{x(t)\} = \int_0^{\infty} x(t)e^{-st} dt \quad (1)$$

où  $s = \sigma + j\omega$  est une variable complexe composée de :

- $\sigma$ , qui traduit l'amortissement ou la croissance du signal
- $\omega$ , qui représente la fréquence du signal en radians par seconde

L'un des principaux intérêts de cette transformation est que les dérivées deviennent de simples multiplications par  $s$ . Ainsi,

$$\frac{d}{dt} \longrightarrow s \quad (2)$$

Cette propriété simplifie considérablement l'étude des systèmes dynamiques et permet notamment d'analyser leur stabilité, leur rapidité ainsi que leur comportement en fréquence.

## Fonction de transfert

Une fois le système exprimé dans le domaine de Laplace, il est possible de définir sa fonction de transfert. La fonction de transfert représente le rapport entre la sortie et l'entrée du système :

$$H(s) = \frac{Y(s)}{X(s)} \quad (3)$$

où :

- $X(s)$  est la transformée de Laplace du signal d'entrée
- $Y(s)$  est la transformée de Laplace du signal de sortie

La fonction de transfert caractérise complètement le comportement dynamique d'un système linéaire lorsque les conditions initiales sont nulles. Elle permet notamment de déterminer la stabilité d'un système, sa rapidité de réponse ainsi que son comportement vis-à-vis des différentes fréquences présentes dans un signal.

Pour faciliter l'implémentation de différents types de filtres, nous avons créé une classe abstraite `IFilter` qui définit une interface commune pour tous les filtres. Cette classe contient deux méthodes abstraites :

- `float apply(float value, float dt)` : cette méthode prend en entrée une valeur à filtrer et un intervalle de temps (`dt`) et retourne la valeur filtrée.
- `void reset()` : cette méthode réinitialise l'état interne du filtre, ce qui est utile pour les filtres qui maintiennent un historique des valeurs précédentes

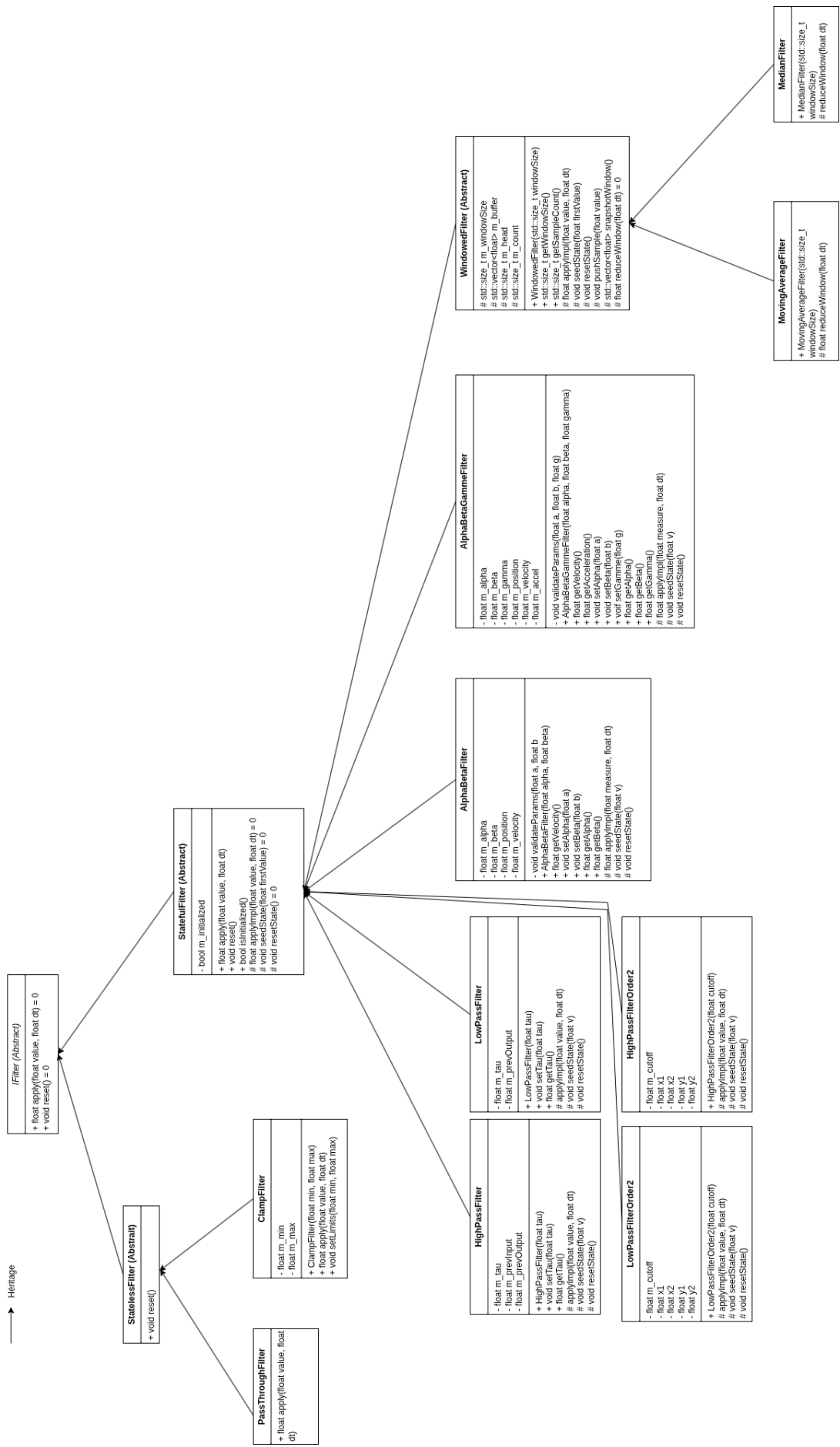


Figure 3: Diagramme de classes des filtres

Nous avons aussi la classe abstraite **StatufulFilter** qui hérite de **IFilter** et qui est destinée aux filtres qui nécessitent de conserver un état interne, comme le filtre passe-bas et passe-haut. Cette classe contient des méthodes pour gérer l'état interne du filtre telles que l'initialisation, la mise à jour et la réinitialisation de l'état.

Il y a aussi la classe abstraite **WindowedFilter** qui hérite de **StatufulFilter** et qui est destinée aux filtres qui utilisent une fenêtre glissante pour traiter les données, comme le filtre médian. Cette classe contient des méthodes pour gérer la fenêtre glissante, telles que l'ajout de nouvelles valeurs, le calcul sur la fenêtre et la réinitialisation de la fenêtre.

Nous allons nous pencher plus en détail sur les filtres passe-bas et médian, qui sont les deux types de filtres que nous avons décidé d'analyser parmi les différents types de filtres existants.

### 5.1.1 Le filtre passe-Bas (ordre 1)

Le filtre passe-bas est le filtre le plus utilisé en automatique. Son objectif est d'atténuer les composantes de haute fréquence du signal tout en conservant les variations lentes représentant généralement l'information utile.

Dans le domaine de Laplace, la fonction de transfert d'un filtre passe-bas du premier ordre est donnée par :

$$H(s) = \frac{1}{\tau s + 1} \quad (4)$$

où :

- $\tau$  représente la constante de temps du filtre
- $s$  est la variable complexe de Laplace

A partir de cette fonction de transfert, il est possible de retrouver l'équation différentielle décrivant le comportement du filtre :

$$\tau \frac{dy(t)}{dt} + y(t) = x(t) \quad (5)$$

avec :

- $x(t)$  le signal d'entrée
- $y(t)$  le signal filtré

Cette équation montre que la sortie ne suit pas instantanément l'entrée. Le filtre agit comme une moyenne glissante : lorsque l'entrée varie brutalement, la sortie évolue progressivement vers cette nouvelle valeur.

Comme le système développé fonctionne de manière discrète, cette équation est discrétisée en utilisant la méthode d'Euler arrière, conduisant aux équations suivantes :

$$\alpha = \frac{\tau}{\tau + dt} \quad (6)$$

$$y[n] = \alpha y[n-1] + (1 - \alpha)x[n] \quad (7)$$

où :

- $y[n]$  est la sortie filtrée à l'instant  $n$
- $x[n]$  est l'entrée à l'instant  $n$

- $y[n - 1]$  est la sortie filtrée à l'instant  $n - 1$
- $\alpha$  est le coefficient de filtrage, qui dépend de la constante de temps  $\tau$  et de l'intervalle de temps  $dt$
- $dt$  est l'intervalle de temps entre deux échantillons
- $\tau$  est la constante de temps du filtre

Le coefficient  $\alpha$  contrôle directement le comportement du filtre.

- Si  $\alpha$  est proche de 1, le filtre utilise principalement les anciennes valeurs. Le filtrage est fort mais le retard augmente
- Si  $\alpha$  est proche de 0, le filtre suit rapidement le signal d'entrée mais atténué moins efficacement le bruit.

A chaque appel de la fonction `apply()`, une seule nouvelle mesure est nécessaire, ainsi que la sortie calculée à l'itération précédente. Le filtre possède donc une mémoire minimale, ce qui en fait un algorithme très léger en calcul.

## Evaluation

Pour l'évaluation des performances du filtre passe-bas, on utilisera deux signaux, celui désiré et le bruit. Les valeurs sont générées comme ceci :

$$y = \sin(2\pi f_{low}\Delta t) + \sin(2\pi f_{noise}\Delta t) \quad (8)$$

où :

- $f_{low}$  est la fréquence du signal désiré, ici de 10
- $f_{noise}$  est la fréquence du bruit, ici 200
- $\Delta t$  est l'intervalle de temps entre deux échantillons, ici 0.001

Le  $\tau$  sera égal à  $\frac{1}{2\pi f_c}$ , avec  $f_c$  la fréquence de coupure du filtre, ici 15.

Le signal temporel (cf. figure 4) montre que le filtre passe-bas atténue efficacement les oscillations rapides présentes sur le signal brut. La composante basse fréquence est correctement conservée et devient nettement plus visible après filtrage.

On remarque cependant que le signal filtré présente encore quelques oscillations résiduelles autour de la composante utile. Ce comportement est directement lié à la pente d'atténuation du filtre.

Une pente de 20 dB/décade signifie que lorsque la fréquence est multipliée par 10 (par exemple de 20 Hz à 200 Hz ou de 100 Hz à 1000 Hz), le gain du filtre diminue de 20 dB. Autrement dit, les hautes fréquences sont progressivement atténuées, mais pas supprimées instantanément.

Cette pente constitue une propriété fondamentale des filtres du premier ordre. Elle résulte directement de leur fonction de transfert, qui ne possède qu'un seul pôle. Par conséquent, les fréquences situées juste après la fréquence de coupure restent encore partiellement présentes dans le signal, ce qui explique que le bruit observé ne disparaisse pas complètement après filtrage.

### Remarque

Le filtrage introduit également un léger retard temporel entre le signal brut et le signal filtré, phénomène inhérent à tous les filtres causaux.

### Origine de la pente d'atténuation

Pour un filtre du premier ordre, le gain décroît approximativement comme

$$|H(j\omega)| \propto \frac{1}{\omega}$$

aux hautes fréquences. En exprimant ce gain en décibels,

$$G_{dB} = 20 \log_{10} |H(j\omega)|$$

on montre que chaque multiplication de la fréquence par dix entraîne une diminution du gain de 20 dB. C'est pourquoi un filtre du premier ordre possède une pente de -20 dB/décade.

L'analyse fréquentielle (cf. figure 5) confirme ce comportement. Le spectre brut est constitué de deux composantes principales :

- une composante utile située à 10 Hz
- une composante parasite située à 200 Hz

Après filtrage :

- la composante de 10 Hz est quasiment conservée
- la composante de 200 Hz est fortement atténuée mais reste visible

Le filtre réalise donc bien sa fonction de passe-bas mais ne supprime pas complètement les hautes fréquences.

La densité spectrale de puissance (PSD) (cf. figure 38) confirme aussi ces observations. La puissance associée aux hautes fréquences diminue fortement après filtrage. Toutefois, une partie de l'énergie demeure présente autour de 200 Hz. Cette observation est cohérente avec les propriétés théoriques du filtre passe-bas.

### Avantages

- Coût de calcul très faible
- Implémentation simple
- Parfaitement adapté aux systèmes embarqués

### Inconvénients

- Retard introduit sur la mesure
- Pente d'atténuation limitée à 20 dB/décade
- Performances limitées lorsque le bruit possède une énergie importante proche de la fréquence de coupure



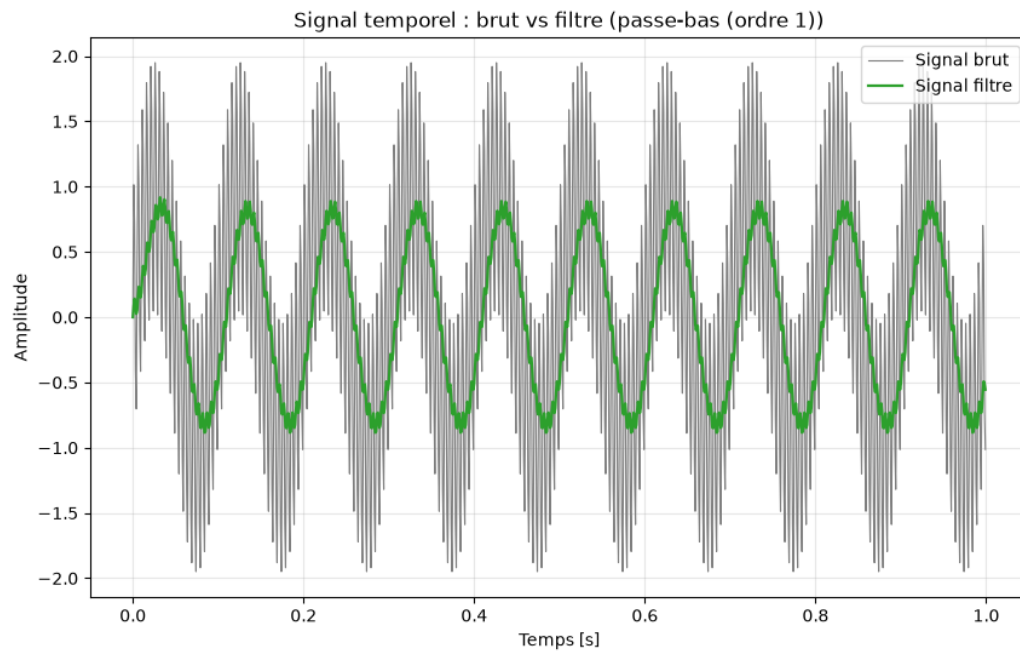


Figure 4: Résultat du filtre passe-bas d'ordre 1 sur des fréquences avec du bruit

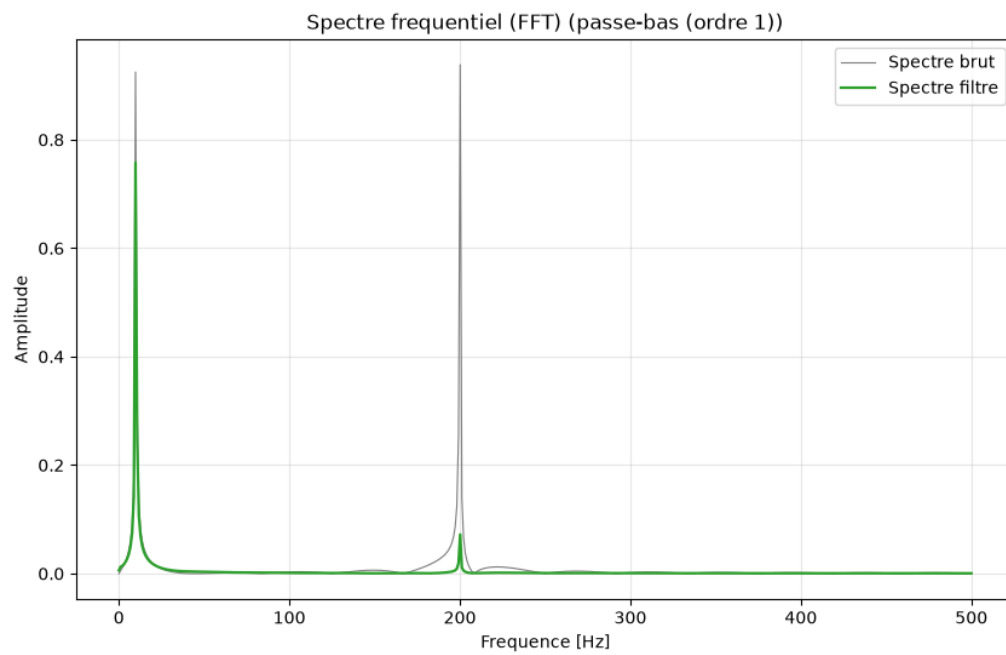


Figure 5: Spectre fréquentiel du signal avant et après filtre passe-bas d'ordre 1

### 5.1.2 Le filtre passe-Haut (ordre 1)

Le filtre passe-haut réalise l'opération inverse.

Son objectif est de supprimer les composants lents du signal, telles que les dérives ou les offsets des capteurs, afin de ne conserver que les variations rapides.

Sa fonction de transfert est :

$$H(s) = \frac{\tau s}{1 + \tau s} \quad (9)$$

Cette fonction conduit à l'équation :

$$\tau \frac{dy(t)}{dt} + y(t) = \tau \frac{dx(t)}{dt} \quad (10)$$

Le filtre réagit donc essentiellement aux variations du signal. Après discrétisation, on obtient :

$$\alpha = \frac{\tau}{\tau + \Delta t} \quad (11)$$

$$y[n] = \alpha(y[n-1] + x[n] - x[n-1]) \quad (12)$$

où :

- $y[n]$  est la sortie filtrée à l'instant  $n$
- $x[n]$  est l'entrée à l'instant  $n$
- $y[n-1]$  est la sortie filtrée à l'instant  $n-1$
- $x[n-1]$  est l'entrée à l'instant  $n-1$
- $\Delta t$  est l'intervalle de temps entre deux échantillons
- $\tau$  est la constante de temps du filtre
- $\alpha$  est le coefficient de filtrage, qui dépend de la constante de temps  $\tau$  et de l'intervalle de temps  $\Delta t$

Le filtre mémorise simultanément la dernière valeur d'entrée et la dernière valeur de sortie, ce qui lui permet de réagir aux variations du signal.

### Evaluation

Pour l'évaluation des performances du filtre passe-haut, on va prendre deux signaux, celui désiré et le bruit. les valeurs sont générées comme ceci :

$$y = \sin(2\pi f_{noise}\Delta t) + \sin(2\pi f_{high}\Delta t) \quad (13)$$

où :

- $f_{high}$  est la fréquence du signal désiré, ici de 100
- $f_{noise}$  est la fréquence du bruit, ici 10
- $\Delta t$  est l'intervalle de temps entre deux échantillons, ici 0.001

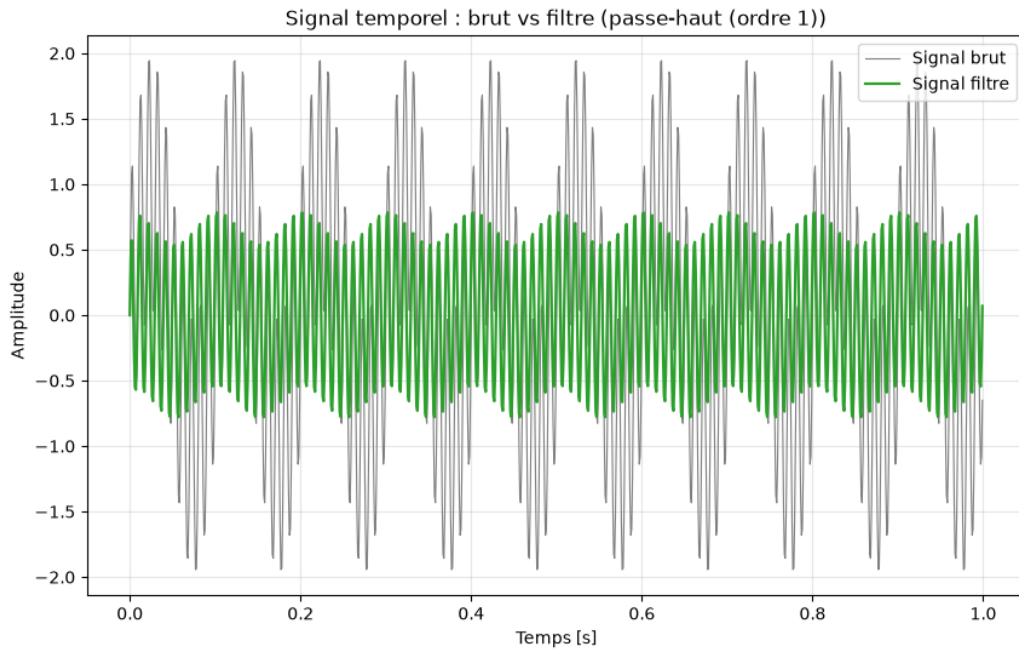


Figure 6: Résultat du filtre passe-haut d'ordre 1 sur des fréquences avec du bruit

Le  $\tau$  sera égal à  $\frac{1}{2\pi f_c}$ , avec  $f_c$  la fréquence de coupure du filtre, ici 80.

Le filtre passe-haut élimine efficacement une grande partie des composantes de basse fréquence présentes dans le signal. Après filtrage, seule la sinusoïde de haute fréquence reste visible (cf. figure 6).

L'analyse fréquentielle (cf. figure 7) confirme ce comportement. Le spectre brut est constitué de deux composantes principales :

- une composante parasite située à 10 Hz
- une composante utile située à 100 Hz

Après filtrage :

- la composante de 10 Hz est fortement réduite
- la composante de 100 Hz est quasiment conservée

Toutefois, un faible résidu subsiste à basse fréquence.

La densité spectrale de puissance (cf. figure 39) confirme aussi ces observations. La puissance des basses fréquences diminue fortement après filtrage. Toutefois, une partie de l'énergie demeure présente autour de 10 Hz. Cette observation est cohérente avec les propriétés théoriques du filtre passe-haut.

### Avantages

- Suppression des offsets
- Élimination des dérives lentes

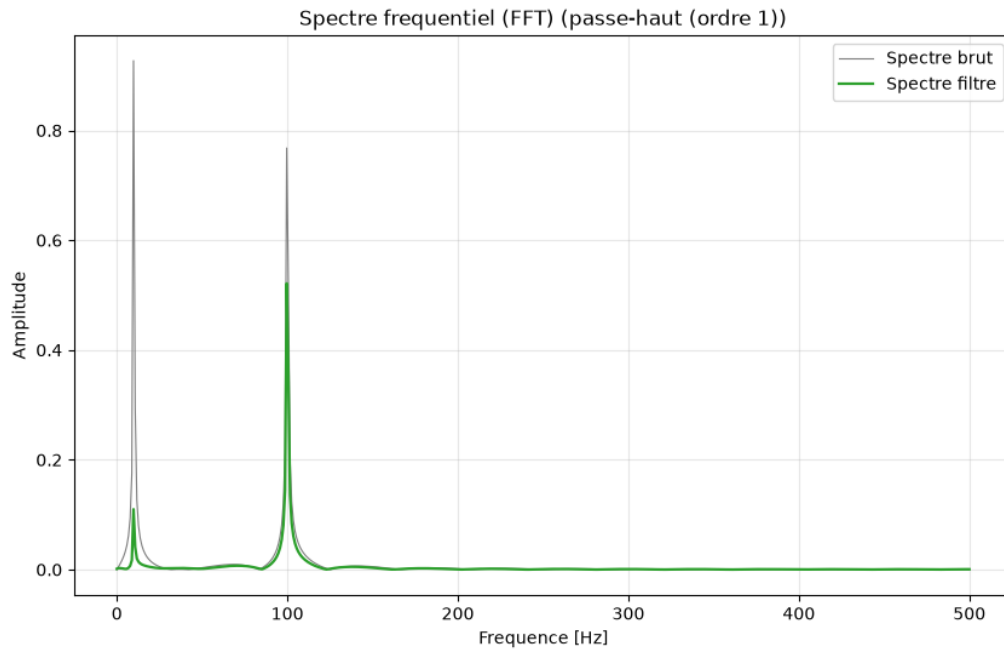


Figure 7: Spectre fréquentiel du signal avant et après filtre passe-haut d'ordre 1

### Inconvénients

- Amplification du bruit haute fréquence
- Rarement utilisé seul pour filtrer une mesure.

#### 5.1.3 Le filtre Butterworth (ordre 2)

Comme expliqué précédemment, un filtre du premier ordre possède une pente asymptotique de  $-20$  dB/décade, ce qui signifie que son atténuation augmente relativement lentement lorsque la fréquence dépasse la fréquence de coupure. Les composantes fréquentielles proches de cette fréquence restent donc encore partiellement présentes dans le signal. Pour améliorer les performances du filtrage, il est nécessaire d'augmenter l'ordre du filtre.

Un filtre d'ordre 2 possède deux pôles. Chaque pôle apportant une pente supplémentaire de  $20$  dB/décade, un filtre du second ordre atteint une pente de  $-40$  dB/décade, ce qui permet une atténuation beaucoup plus rapide des hautes fréquences. Parmi les différentes familles existantes, les filtres Butterworth constituent un excellent compromis entre simplicité, stabilité et qualité de filtrage.

Le filtre de Butterworth a été proposé par Stephen Butterworth en 1930 afin d'obtenir une réponse fréquentielle maximale plate dans la bande passante. Cette propriété signifie que le gain reste pratiquement constant jusqu'à la fréquence de coupure, sans oscillation ni ondulation, ce qui permet de préserver les informations utiles du signal tout en améliorant fortement l'atténuation des hautes fréquences.

Un filtre de Butterworth d'ordre 2 a été développé pour remplacer le filtre passe-bas du premier ordre et passe-haut du premier ordre.

Sa fonction de transfert est :

$$H(s) = \frac{\omega_c^2}{s^2 + \sqrt{2}\omega_c s + \omega_c^2} \quad (14)$$

$$\omega_c = 2\pi f_c \quad (15)$$

Contrairement aux filtres précédents, cette fonction possède deux pôles. Le filtre dépend donc non seulement de la mesure actuelle mais également des deux mesures précédentes ainsi que des deux sorties précédentes.

Après discrétisation, l'équation générale devient :

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2] \quad (16)$$

Les coefficients  $a_i$  et  $b_i$  sont recalculés à partir de la fréquence de coupure, de l'intervalle de temps entre deux mesures et du facteur de qualité  $Q = \frac{1}{\sqrt{2}}$  qui correspond précisément au filtre de Butterworth.

Contrairement au filtre du premier ordre, le filtre utilise quatre états internes, qui sont les deux anciennes entrées et deux anciennes sorties. Cette mémoire supplémentaire lui permet d'obtenir une pente d'atténuation de 40 dB/décade, soit deux fois plus importante que celle du filtre du premier ordre.

Ce qui change entre le filtre passe-haut et passe-bas d'ordre 2, ce sont les coefficients  $a_i$  et  $b_i$  qui sont recalculés en fonction de la fréquence de coupure et du type de filtre.

Pour le filtre passe-bas :

$$\begin{cases} a_0 = 1 + \alpha \\ a_1 = \frac{-2\cos(\omega)}{a_0} \\ a_2 = \frac{1-\alpha}{a_0} \\ b_0 = \frac{\frac{1-\cos(\omega)}{2}}{a_0} \\ b_1 = \frac{1-\cos(\omega)}{a_0} \\ b_2 = \frac{\frac{1-\cos(\omega)}{2}}{a_0} \end{cases} \quad (17)$$

Pour le filtre passe-haut :

$$\begin{cases} a_0 = 1 + \alpha \\ a_1 = \frac{-2\cos(\omega)}{a_0} \\ a_2 = \frac{1-\alpha}{a_0} \\ b_0 = \frac{(1+\cos(\omega))*0.5}{a_0} \\ b_1 = \frac{-(1+\cos(\omega))}{a_0} \\ b_2 = \frac{(1+\cos(\omega))*0.5}{a_0} \end{cases} \quad (18)$$

où :

$$\alpha = \frac{\sin(\omega)}{2 * Q} \omega = \frac{2\pi * f_c}{f_s} f_s = \frac{1}{\Delta t} \quad (19)$$

#### Remarque

Pour les évaluations, nous avons choisi de conserver les mêmes signaux que pour les filtres du premier ordre ainsi que des mêmes fréquences de coupure. Cela permet de comparer directement les performances des filtres d'ordre 1 et d'ordre 2.

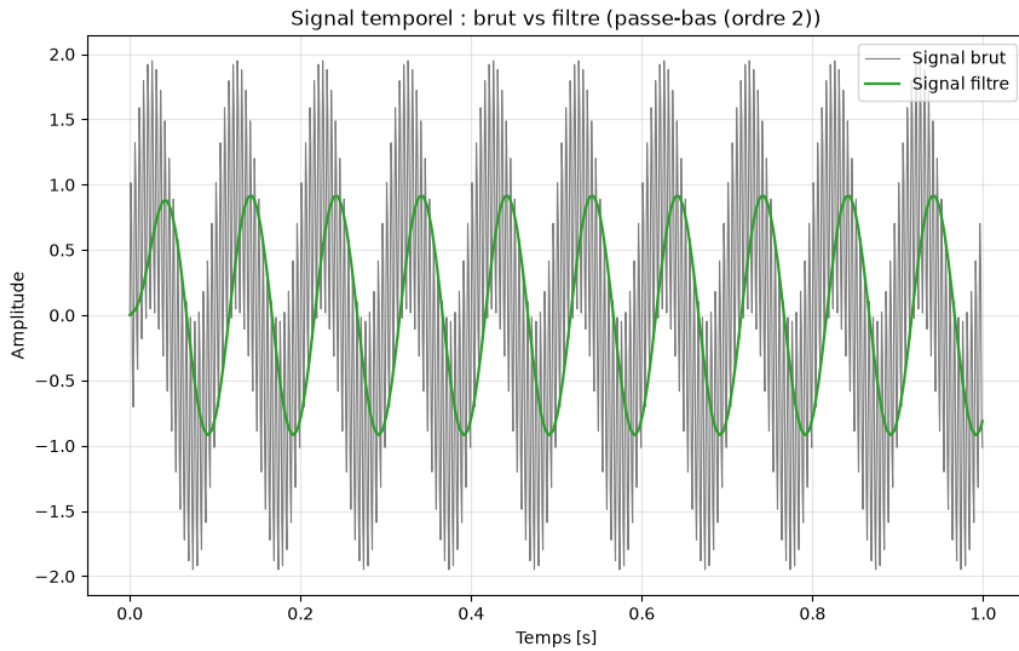


Figure 8: Résultat du filtre passe-bas d'ordre 2 sur des fréquences avec du bruit

### Evaluation du filtre passe-bas Butterworth d'ordre 2

Le signal filtré obtenu avec le filtre de Butterworth apparaît beaucoup plus lisse. Les oscillations rapides ont pratiquement disparu et la sinusoïde basse fréquence est conservée avec une très bonne fidélité (cf. figure 8). Par rapport au filtre d'ordre 1, la composante utile est plus propre et présente moins de fluctuations parasites. On observe néanmoins un retard légèrement supérieur, conséquence directe de l'augmentation de l'ordre du filtre.

Le spectre fréquentiel met clairement en évidence les performances du filtre (cf. figure 5) où la composante utile de 10 Hz est quasiment inchangée. En revanche, la composante de 200 Hz est presque entièrement supprimée. Cette différence est directement liée à la pente d'atténuation de 40 dB/décade du filtre de Butterworth.

La densité spectrale de puissance (cf. figure 40) confirme l'amélioration, la puissance du bruit haute fréquence est nettement plus faible que pour le filtre d'ordre 1. Au-delà de 250 Hz, l'énergie résiduelle ne devient pratiquement nulle. Le filtre élimine donc beaucoup plus efficacement les hautes fréquences.

### Evaluation du filtre passe-haut Butterworth d'ordre 2

Le signal filtré obtenu avec le filtre passe-haut de Butterworth d'ordre 2 est beaucoup plus régulier que celui obtenu avec le filtre d'ordre 1 (cf. figure 10). La composante basse fréquence est pratiquement absente, la sinusoïde haute fréquence est conservée avec une amplitude presque constante. Contrairement au filtre d'ordre 1, on ne distingue pratiquement plus de modulation d'amplitude. Cela traduit une meilleure suppression des basses fréquences.

Le spectre fréquentiel (cf. figure 11) montre une différence particulièrement nette entre les deux filtres. La composante de 10 Hz disparaît presque complètement, tandis que la composante

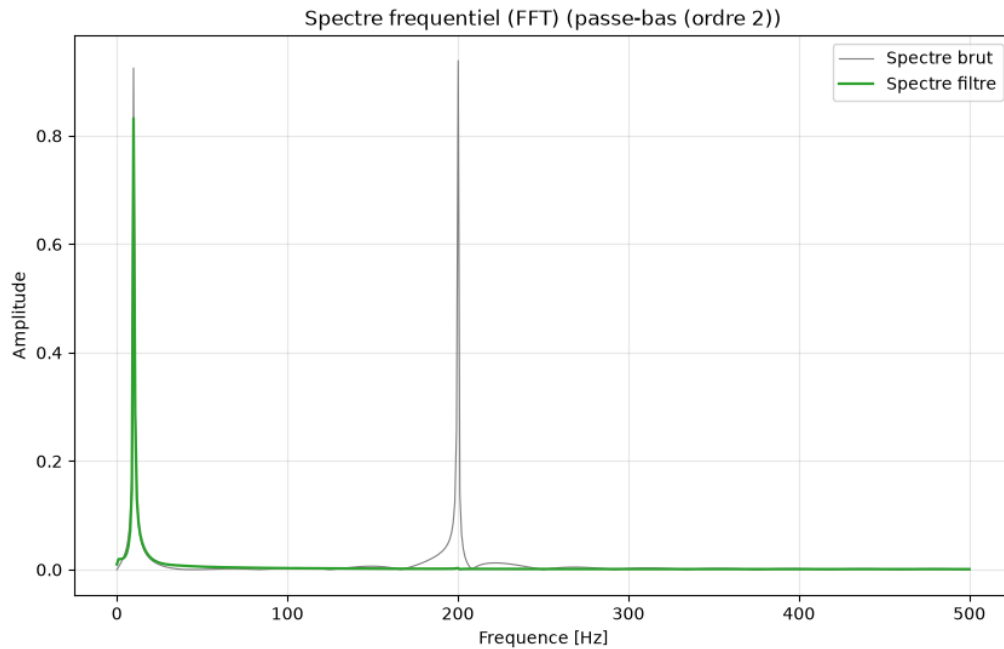


Figure 9: Spectre fréquentiel du signal avant et après filtre passe-bas d'ordre 2

de 100 Hz est parfaitement conservée. Le filtre réalise donc une séparation fréquentielle beaucoup plus efficace.

La densité spectrale de puissance (cf. figure 41) montre une diminution très importante de la puissance des basses fréquences. La composante utile est conservée alors que les fréquences indésirables sont fortement atténuées.

## Conclusion

Le filtre Butterworth peut être considéré comme une évolution naturelle des filtres du premier ordre. Il conserve les mêmes objectifs de filtrage, il préserve beaucoup mieux les fréquences désirées tout en atténuant plus efficacement les fréquences indésirables. Ils ont une excellente stabilité numérique, mais cela a un coût de calcul plus élevé que le filtre du premier ordre.

### 5.1.4 Le filtre médian

#### Principe

Le filtre médian est un filtre non-linéaire qui remplace chaque valeur d'un signal par la médiane des valeurs voisines dans une fenêtre glissante. Contrairement aux filtres linéaires, le filtre médian est particulièrement efficace pour éliminer les pics de bruit tout en préservant les bords et les détails du signal.

#### Fonctionnement

Le filtre médian est de type WindowedFilter, ce qui signifie qu'il utilise une fenêtre glissante pour traiter les données. La taille de cette fenêtre est un paramètre clé du filtre, car elle détermine le nombre de valeurs voisines à considérer pour calculer la médiane. Il fonctionne en 3 étapes :

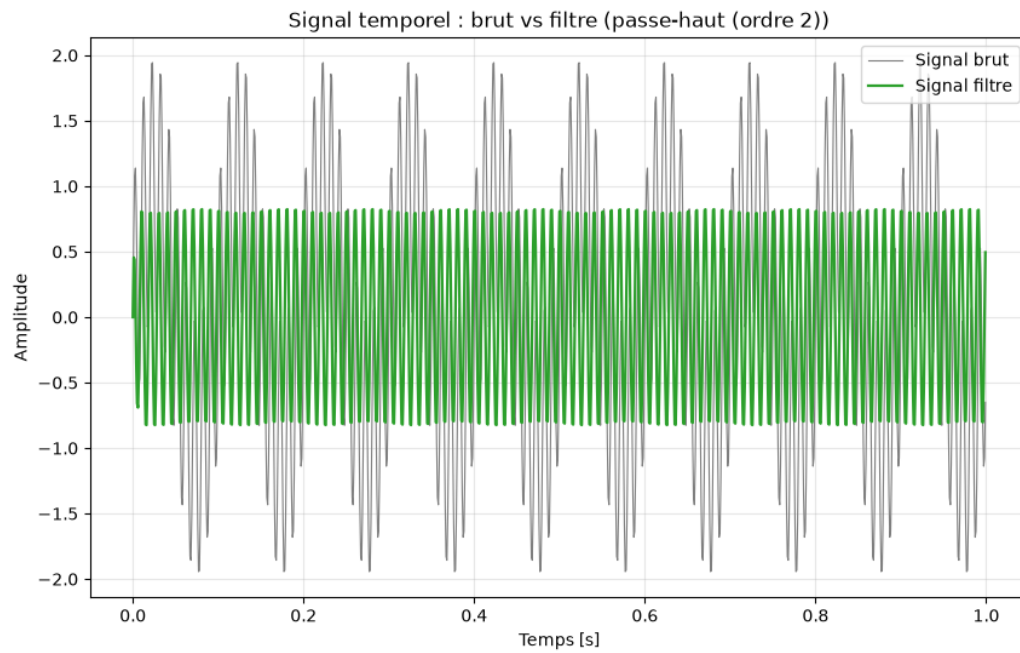


Figure 10: Résultat du filtre passe-haut d'ordre 2 sur des fréquences avec du bruit

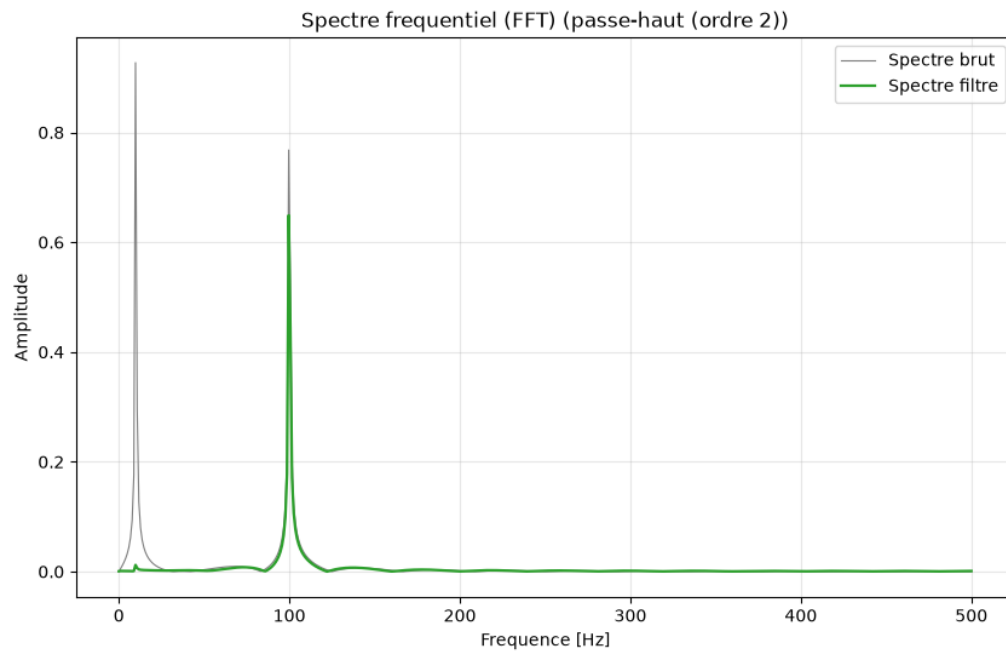


Figure 11: Spectre fréquentiel du signal avant et après filtre passe-haut d'ordre 2



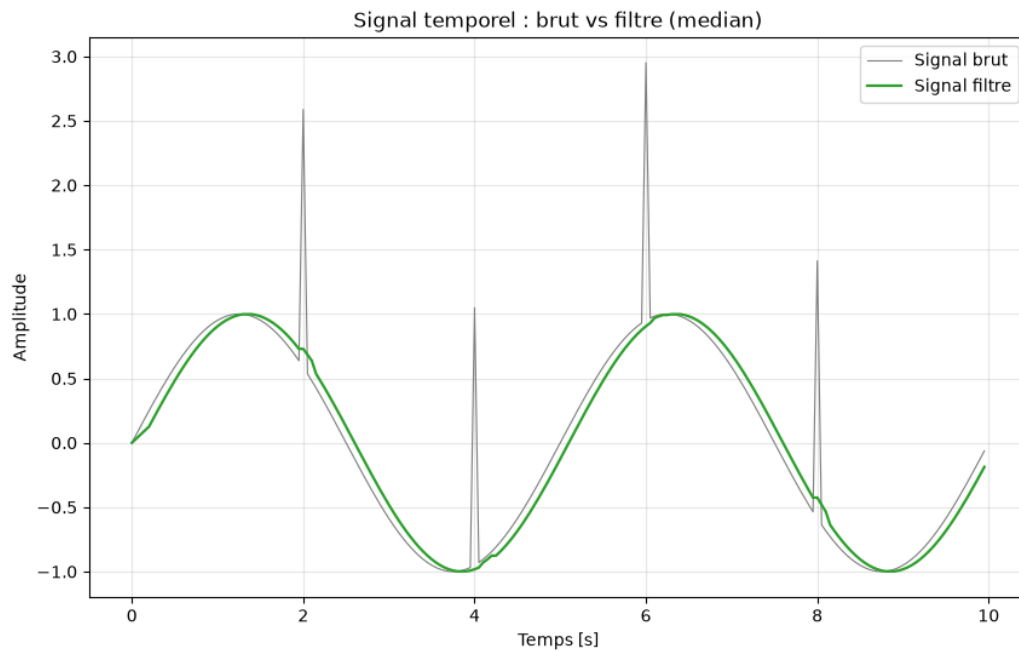


Figure 12: Résultat du filtre médian sur des fréquences avec des pics de bruit, avec une fenêtre de 5

1. Pour chaque point, il ajoute sa valeur dans un tableau de taille fixe (la taille de la fenêtre)
2. Il récupère les valeurs du tableau et les trie par ordre croissant
3. Il retourne la valeur médiane du tableau trié

### Exemple d'utilisation

Dans notre exemple, nous avons utilisé un signal sinusoïdal avec des pics de bruit aléatoires pour illustrer l'effet du filtre médian sur le signal. Le signal est défini par l'équation suivante :

$$y = \sin(2 * \pi * 0.2 * t) + spike$$

Où :

- $y$  est la valeur du signal à l'instant  $t$
- $t$  est le temps en secondes
- $spike$  est un pic de bruit aléatoire qui est égal à 2 toutes les 2 secondes et à 0 sinon

On a appliqué le filtre médian sur ce signal avec une fenêtre de taille 5 et on a obtenu le résultat présenté dans la figure 12. On peut voir que le filtre médian a réussi à éliminer les pics de bruit tout en préservant la forme générale du signal sinusoïdal.

### Avantages et Inconvénients

Le filtre médian présente plusieurs avantages par rapport aux filtres linéaires, notamment sa capacité à éliminer efficacement les pics de bruit tout en préservant les bords et les détails du signal. Cependant, il a également quelques inconvénients, tels qu'en fonction de la taille de la

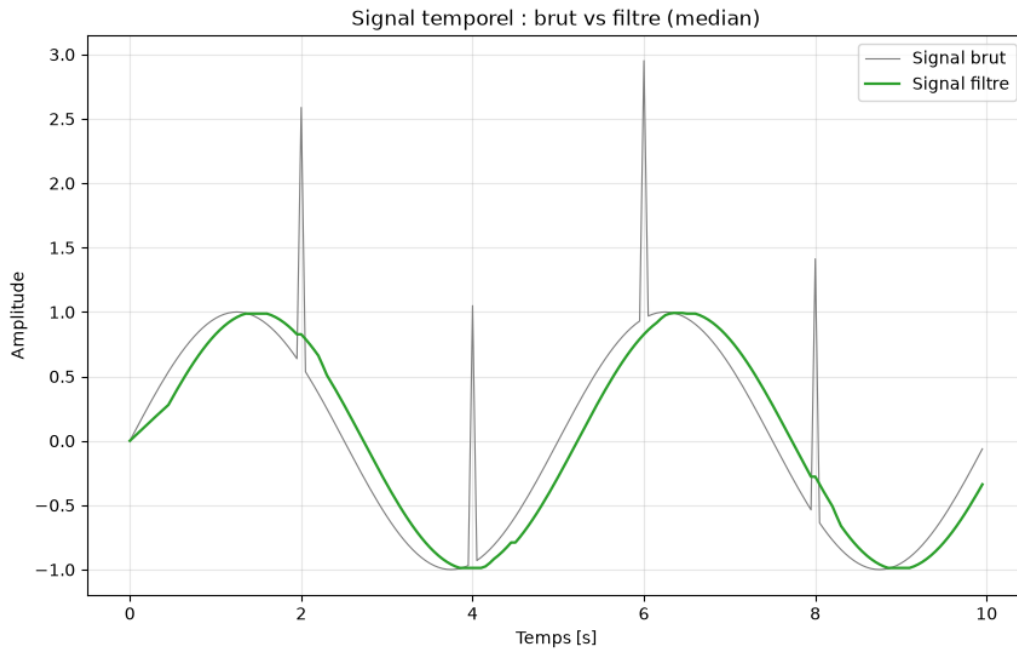


Figure 13: Résultat du filtre médian sur des fréquences avec des pics de bruit, avec une fenêtre de 10

fenêtre, il peut introduire un décalage temporel dans le signal filtré et peut rendre un signal qui était continu en un signal discontinu (cf. figures 13 et 14)

### 5.1.5 Evaluation des filtres sur des signaux d'instruments de musique

Nous allons maintenant utiliser nos filtres passe-bas et passe-haut d'ordre 2 pour filtrer des signaux réalistes provenant d'instruments de musique. Pour notre exemple, nous avons choisi un projet musical issu de la plateforme [Cambridge Music Technology](#) se prénommant "Timeless (Part 2)" de l'artiste MekaPhil. Nous avons extrait les pistes audio de la basse, de la guitare électrique et de la flûte, pour avoir une piste audio mixée contenant les trois instruments<sup>1</sup>.

L'objectif de cette expérimentation est de vérifier si un filtre passe-bande, obtenu par la combinaison d'un filtre passe-bas et un filtre passe-haut, permet d'isoler la bande fréquentielle caractéristique d'un instrument présent dans un signal audio composé de plusieurs instruments. Pour chaque instrument, trois signaux sont comparés :

- la piste originale de l'instrument (référence)
- le signal audio contenant le mélange des instruments
- le signal obtenu après filtrage du mélange.

L'analyse repose alors sur la transformée de Fourier (FFT) et la densité spectrale de puissance (PSD).

Pour chaque instrument, nous avons choisi des fréquences de coupure pour le filtre passe-bas et le filtre passe-haut afin de ne conserver que les fréquences de l'instrument. Les fréquences de

<sup>1</sup>Cette piste audio dure 4 secondes au lieu de 30 secondes pour faciliter les coûts de calcul

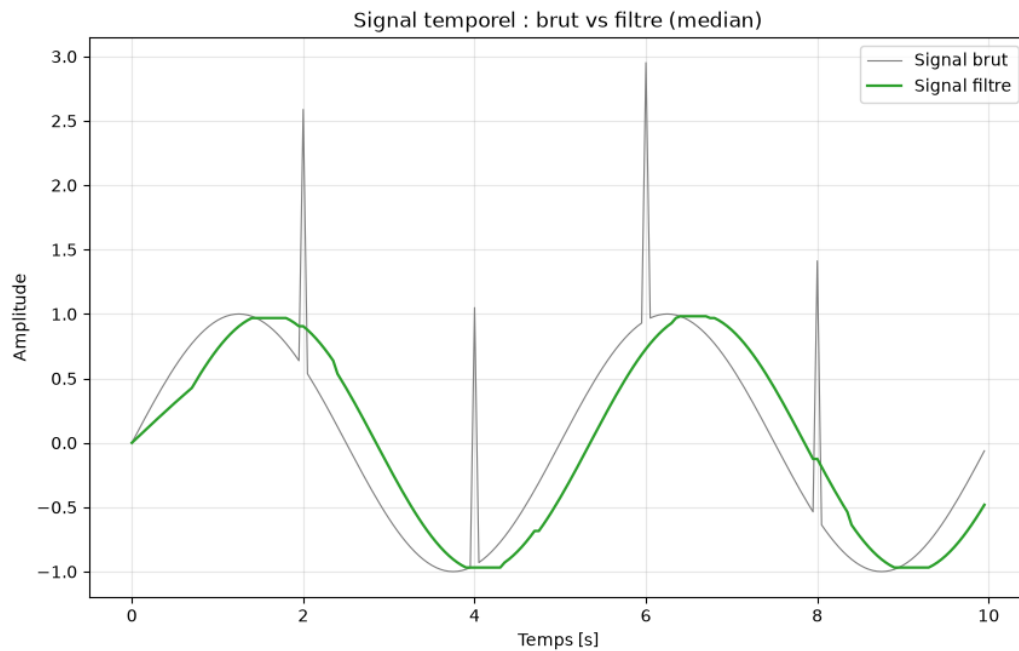


Figure 14: Résultat du filtre médian sur des fréquences avec des pics de bruit, avec une fenêtre de 15

coupure ont été choisies en fonction des caractéristiques fréquentielles de chaque instrument et qui sont les suivantes :

- Basse : 30 Hz à 250 Hz
- Guitare électrique : 280 Hz à 900 Hz
- Flûte : 1000 Hz à 1400 Hz

#### Remarque

Ces fréquences ont été choisies via un [tableau des fréquences des instruments de musique](#) mais aussi via l'analyse fréquentielle de chaque piste audio des instruments seules (cf. figures 15, 17 et 19).

#### Analyse des résultats de la basse

La FFT de la basse seule (cf. figure 15) montre que son énergie est principalement concentrée sous 250 Hz, ce qui est conforme à la littérature concernant les fréquences fondamentales de la basse. Après filtrage du mélange entre 30 Hz et 250 Hz, on observe que :

- les composantes situées dans cette bande sont correctement conservées
- les composantes de moyenne et haute fréquence sont fortement atténuées

En revanche, la FFT du signal filtré montre encore quelques composantes résiduelles provenant d'autres instruments. Cette observation est normale puisque certains instruments produisent également des harmoniques dans les basses fréquences.

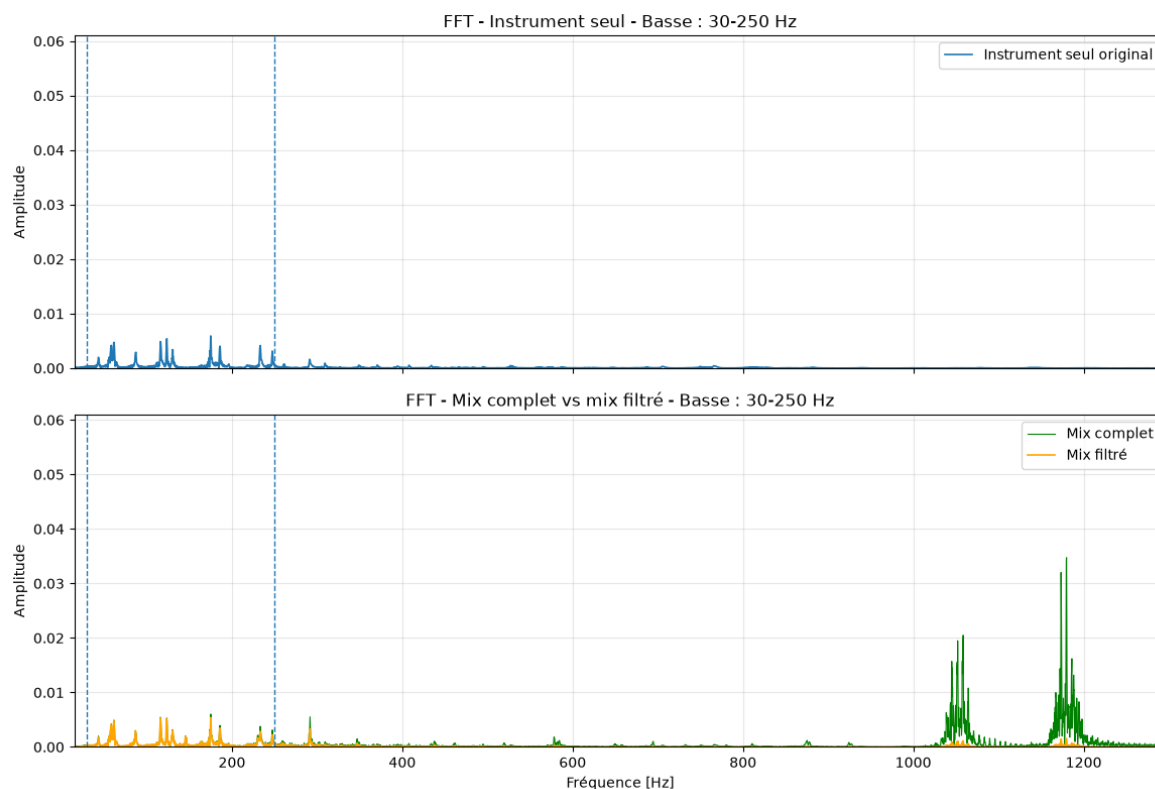


Figure 15: Spectre fréquentiel de la piste audio de la basse

La densité spectrale de puissance (cf. figure 16) confirme ce résultat. La puissance du signal filtré est concentrée dans la bande étudiée, tandis que les fréquences supérieures présentent une forte diminution de puissance. La décroissance importante observée après 250 Hz traduit le bon fonctionnement du passe-bas.

Le filtre permet de mettre en évidence les fréquences caractéristiques de la basse mais ne supprime pas totalement les autres instruments.

### Analyse des résultats de la guitare électrique

La FFT de la guitare électrique seule (cf. figure 17) montre une énergie répartie sur une plage relativement large. Le signal filtré conserve correctement les pics principaux présents dans cette bande. Cependant plusieurs composantes situées au-dessus de 1000 Hz subsistent dans le mélange. Cela s'explique par les nombreuses harmoniques produites par la guitare, mais également par les autres instruments. Le filtre réduit leur influence sans pouvoir les supprimer complètement.

La densité spectrale de puissance (cf. figure 18) montre une forte réduction de puissance en dehors de la bande sélectionnée. En revanche, la puissance dans la bande 280-900 Hz reste très proche de celle observée sur le signal original. Cette observation confirme que le filtre conserve efficacement la partie utile du signal.

La guitare est correctement mise en évidence. Les composantes principales sont conservées alors que les autres fréquences sont largement atténuées.

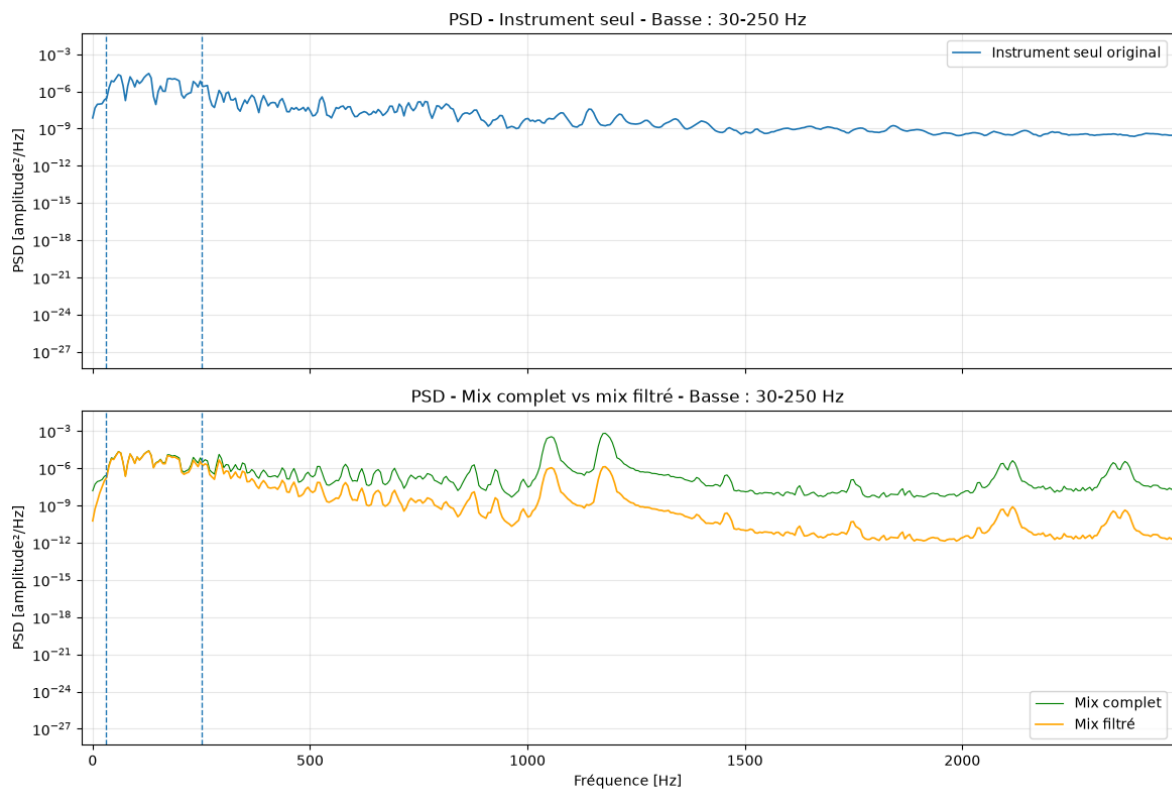


Figure 16: Densité spectrale de puissance de la piste audio de la basse

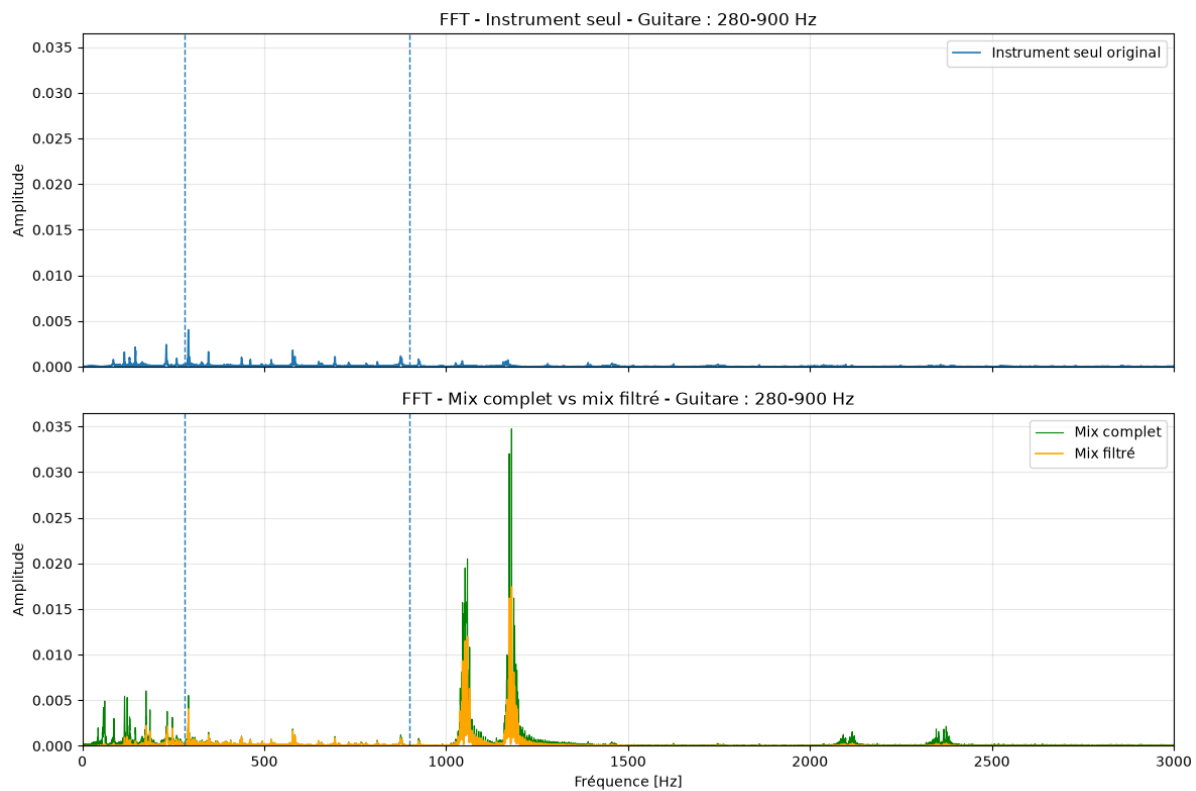


Figure 17: Spectre fréquentiel de la piste audio de la guitare

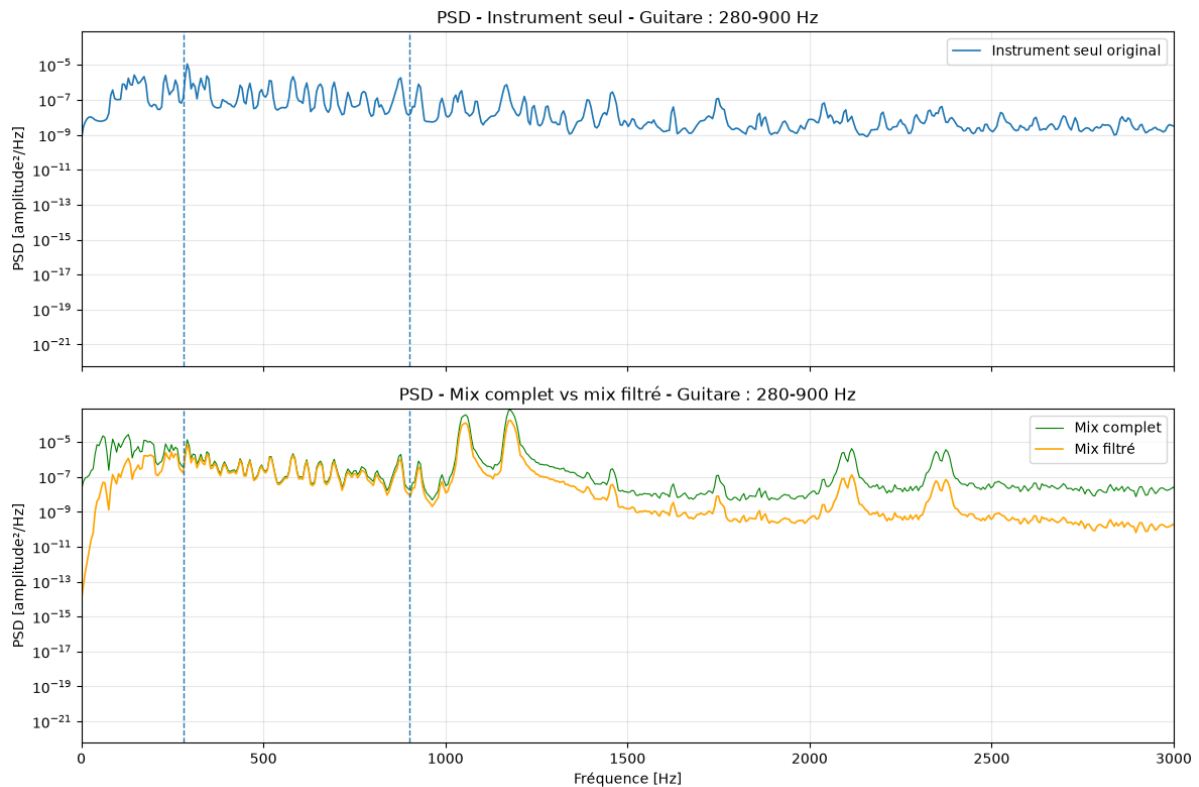


Figure 18: Densité spectrale de puissance de la piste audio de la guitare

### Analyse des résultats de la flûte

La FFT de la flûte seule (cf. figure 19) montre que son énergie est concentrée dans une bande relativement étroite. Le signal filtré conserve correctement les composantes principales situées entre 1000 Hz et 1400 Hz. Cependant, certaines harmoniques situées au-dessus de 1500 Hz subsistent dans le mélange. Cela s'explique par les nombreuses harmoniques produites par la flûte mais également par les autres instruments. Le filtre réduit leur influence sans pouvoir les supprimer complètement.

La densité spectrale de puissance (cf. figure 20) montre une forte réduction de puissance en dehors de la bande sélectionnée. En revanche, la puissance dans la bande 1000-1400 Hz reste très proche de celle observée sur le signal original. Cette observation confirme que le filtre conserve efficacement la partie utile du signal.

La séparation est nettement meilleure que pour la basse et la guitare électrique. Cela s'explique par le fait que la bande fréquentielle utilisée est moins encombrée par les autres instruments du mélange.

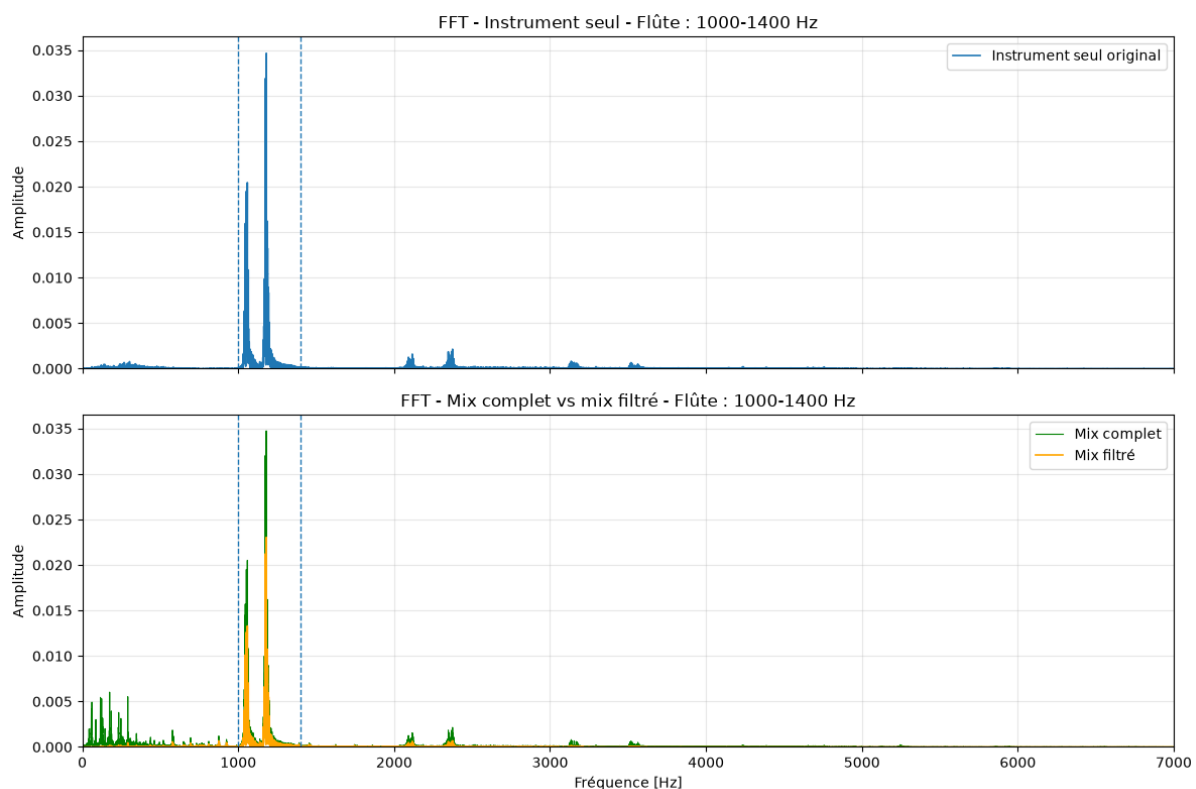


Figure 19: Spectre fréquentiel de la piste audio de la flûte

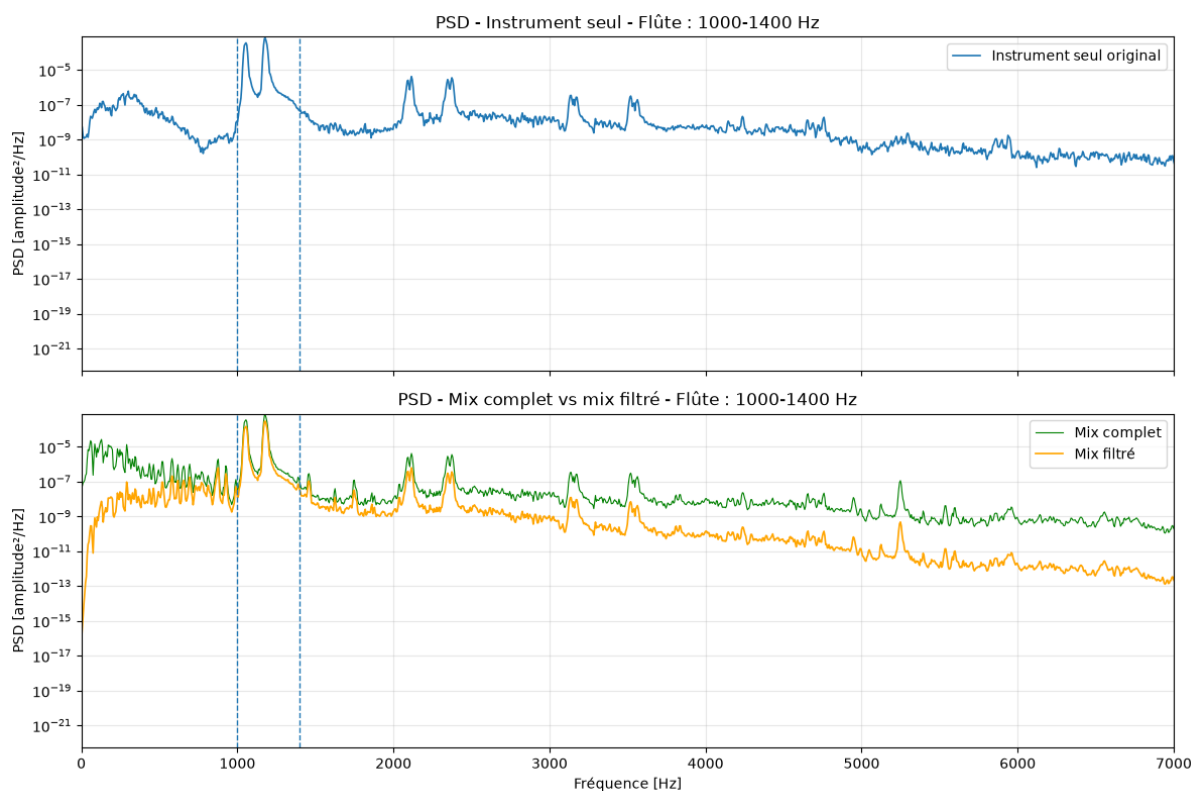


Figure 20: Densité spectrale de puissance de la piste audio de la flûte

## **Analyse globale**

Avec les filtres passe-bas et passe-haut d'ordre 2, on a pu isoler les bandes fréquentielles caractéristiques de chaque instrument. Le filtrage a permis de réduire significativement les composantes indésirables provenant des autres instruments. Cependant, il est important de noter que certains résidus subsistent, en particulier pour la guitare électrique. Cela est dû aux harmoniques générées par les instruments et à la proximité des bandes fréquentielles. Une analyse plus approfondie pourrait être d'utiliser les données récupérées après filtrage et de recréer un signal audio pour écouter le résultat.



## 5.2 L'implémentation des anti-windups

### Remarque

Le calcul de la commande du PID s'effectue avec l'équation suivante :

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (20)$$

où :

- $u(t)$  est le signal de commande
- $e(t)$  est l'erreur (différence entre la consigne et la mesure)
- $K_p$ ,  $K_i$  et  $K_d$  sont respectivement les gains proportionnel, intégral et dérivé du PID

Dans un contrôleur PID, le terme de l'intégral a pour rôle d'éliminer l'erreur statique en accumulant l'erreur au cours du temps. Cependant, lorsque l'actionneur atteint ses limites physiques de fonctionnement, la commande calculée par le contrôleur ne peut plus être appliquée intégralement. On parle alors de saturation de l'actionneur.

Dans cette situation, le terme intégral continue d'accumuler l'erreur alors même que l'actionneur est incapable de produire une commande supplémentaire. Ce phénomène est appelé saturation de l'intégrateur. Lorsque le système quitte la saturation, l'intégrateur contient une valeur excessive qui continue d'influencer la commande pendant un certain temps. Cela peut entraîner un dépassement important de la consigne, des oscillations ainsi qu'une augmentation du temps de stabilisation du système. (cf. figure 21)

L'objectif d'un anti-windup est d'empêcher ou de limiter cette accumulation excessive du terme intégral pendant les phases de saturation. En réduisant l'effet windup, le contrôleur retrouve plus rapidement un fonctionnement normal lorsque la saturation disparaît. Les performances dynamiques du système sont ainsi améliorées, notamment par une diminution du dépassement, une réduction du temps de réponse et une meilleure stabilité globale.

Le principe général d'un anti-windup consiste à modifier le calcul du terme intégral lorsque la commande est saturée. Plusieurs approches existent, parmi lesquelles les plus répandues sont :

- **l'anti-windup clamp**, qui bloque ou limite directement la valeur du terme intégral lorsque celui-ci atteint une borne prédéfinie
- **l'anti-windup back-calculation**, qui corrige progressivement l'intégrateur en fonction de l'écart entre la commande calculée par le PID et la commande réellement appliquée après la saturation

### 5.2.1 L'anti-windup clamp

L'anti-windup clamp consiste à limiter la valeur du terme intégral à une certaine plage. Lorsque la commande calculée par le PID dépasse les limites de l'actionneur, le terme intégral est "clampé" à une valeur maximale ou minimale prédéfinie. Cela empêche l'intégrateur de continuer à accumuler l'erreur et de provoquer un dépassement excessif lorsque la saturation disparaît.

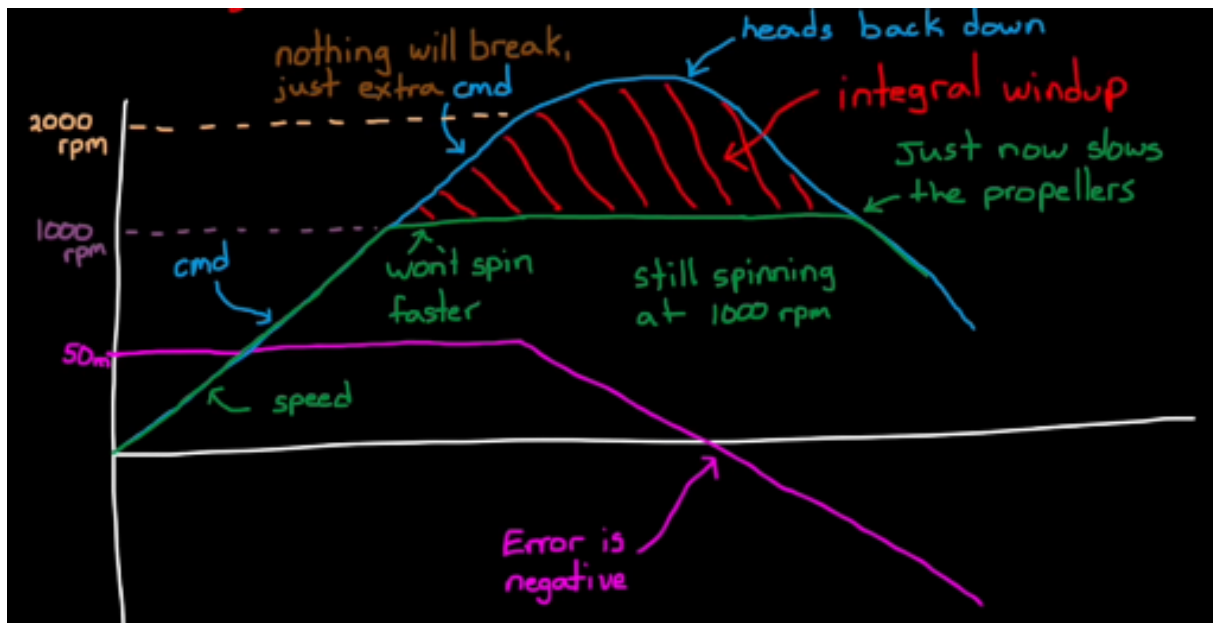


Figure 21: Principe de l'anti-windup sur l'intégral du PID

Le terme intégral est calculé comme suit :

$$I_k = I_{k-1} + K_i e_k \Delta t \quad (21)$$

où :

- $I_k$  est la valeur du terme intégral à l'instant  $k$
- $K_i$  est le gain intégral du PID
- $e_k$  est l'erreur à l'instant  $k$
- $\Delta t$  est le pas de temps de l'échantillonnage

Après la saturation de l'actionneur, le terme intégral est limité à une valeur maximale ou minimale prédéfinie :

$$I_k = \begin{cases} I_{max}, & \text{si } I_k > I_{max} \\ I_{min}, & \text{si } I_k < I_{min} \\ I_k, & \text{sinon} \end{cases} \quad (22)$$

### Avantages

L'anti-windup clamp présente plusieurs avantages :

- Il est simple à implémenter et à configurer
- Il permet de limiter rapidement l'accumulation du terme intégral lors de la saturation
- Il améliore la stabilité du système en évitant les dépassements excessifs

### Inconvénients

Il présente également quelques inconvénients :

- Il peut introduire des discontinuités dans la commande, ce qui peut provoquer des oscillations ou des comportements indésirables

- Il nécessite de définir des bornes appropriées pour le terme intégral, ce qui peut être difficile dans certains systèmes
- Il peut ne pas être efficace dans les systèmes avec des dynamiques complexes ou des non-linéarités importantes

### 5.2.2 L'anti-windup back-calculation

L'anti-windup back-calculation est une approche plus élaborée qui ne consiste pas à limiter l'intégrateur directement, mais à chercher à le corriger afin qu'il reflète la commande effectivement appliquée au système.

Soit :

- $u$  la commande calculée par le PID
- $u_{sat}$  la commande après saturation

La différence  $u_{sat} - u$  représente l'erreur introduite par la saturation. Le principe de back-calculation consiste à réinjecter cette erreur dans l'intégrateur :

$$I_k = I_{k-1} + K_i e_k \Delta t + K_{aw}(u_{sat_k} - u_k) \quad (23)$$

où :

- $K_{aw}$  est le gain de l'anti-windup
- $u_k$  est la commande calculée
- $u_{sat_k}$  est la commande réellement envoyée à l'actionneur

Lorsque le contrôleur est saturé, la différence ( $u_{sat} - u$ ) devient importante. Le terme correcteur agit alors sur l'intégrateur afin de réduire progressivement son accumulation ; Dès que le système quitte la saturation, l'intégrateur possède déjà une valeur proche de celle nécessaire au fonctionnement normal du contrôleur.

Le paramètre essentiel de cette méthode est le gain  $K_{aw}$ . Un gain trop faible rend la correction peu efficace, tandis qu'un gain trop élevé peut introduire des oscillations ou dégrader la stabilité du système. En pratique, il est souvent choisi sous la forme :

$$K_{aw} = \frac{1}{T_t} \quad (24)$$

où  $T_t$  est une constante de temps d'anti-windup réglée en fonction de la dynamique du système.

### Avantages

Il présente plusieurs avantages :

- Il possède une récupération rapide après une saturation
- Il réduit fortement le dépassement
- Il est plus fidèle au comportement réel de l'actionneur

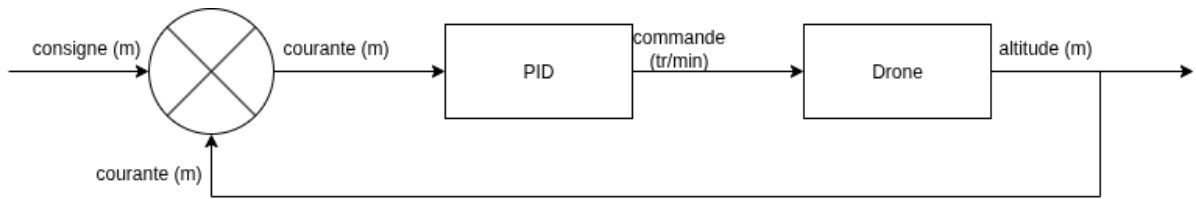


Figure 22: Schéma bloc fonctionnel du système de contrôle du drone

## Inconvénients

Il présente également quelques inconvénients :

- Il est plus complexe à implémenter et à configurer
- Il nécessite un réglage précis du gain  $K_{aw}$  pour éviter les oscillations ou la dégradation de la stabilité
- Il peut être sensible aux variations de la dynamique du système ou aux perturbations externes

### 5.2.3 Illustration par un cas pratique

#### Point important

Dans cette section, il faut bien comprendre que tout est dans un contexte de simulation, et que les résultats obtenus ne sont pas forcément représentatifs de la réalité. Cette section a pour but de montrer l'impact de l'anti-windup sur un système de contrôle, et non pas de fournir une solution optimale pour le contrôle d'un drone.

Pour tester l'efficacité de l'anti-windup, nous avons choisi un cas pratique simple mais représentatif : le contrôle d'altitude d'un drone en fonction de la vitesse de rotation de ses hélices, en suivant l'exemple donné dans la vidéo [Anti-windup for PID control | Understanding PID Control, Part 2](#). L'objectif est de maintenir le drone à une altitude cible en ajustant la vitesse de rotation des hélices (cf. listing 2).

Pour réussir cela, nous avons mis en place un contrôleur PID qui calcule la commande à envoyer aux hélices en fonction de l'erreur entre l'altitude cible et l'altitude actuelle du drone. Cependant, la vitesse de rotation des hélices est limitée à 15 tours/minute, ce qui peut entraîner une saturation de la commande lorsque le drone est loin de l'altitude cible. Dans ce cas, l'intégrateur du PID peut continuer à accumuler l'erreur, ce qui peut provoquer un dépassement important de l'altitude cible lorsque le drone quitte la saturation.

Dans notre scénario, nous avons simulé une situation où le drone est initialement à une altitude de 0 mètre et doit atteindre une altitude cible de 5 mètres. Cependant, nous avons introduit une retenue de 10 secondes avant que le drone ne commence à monter, ce qui provoque une accumulation importante de l'erreur dans l'intégrateur du PID.

Nous avons donc mis en place deux types d'anti-windup : le clamp et le back-calculation, afin de comparer leurs performances dans ce scénario.

### 5.2.4 La configuration du système de contrôle

Le système de contrôle est composé d'un contrôleur PID qui reçoit en entrée l'erreur entre l'altitude cible et l'altitude actuelle du drone, et qui calcule la commande à envoyer aux hélices

pour corriger cette erreur. Le contrôleur PID est configuré avec les paramètres suivants :

$$K_p = 1.5, \quad K_i = 0.6, \quad K_d = 0 \quad (25)$$

Le système est simulé avec un pas de temps de 0.1s, et l'altitude cible est fixée à 5 mètres. Le drone est initialement à une altitude de 0 mètre, et la vitesse de rotation des hélices est limitée à 15 tours/minute.

Le drone est modélisé par les équations suivantes :

$$\begin{cases} a = \frac{F_p}{m} - g - \frac{C_d v}{m} \\ v(t + \Delta t) = v(t) + a \Delta t \\ h(t + \Delta t) = h(t) + v \Delta t \end{cases} \quad (26)$$

Où :

- $a$  est l'accélération verticale du drone (m/s<sup>2</sup>)
- $F_p$  est la force de poussée générée par les hélices (N)
- $m$  est la masse du drone (kg)
- $g$  est l'accélération gravitationnelle (m/s<sup>2</sup>)
- $C_d$  est le coefficient de frottement de l'air
- $v$  est la vitesse verticale du drone (m/s)
- $h$  est l'altitude du drone (m)
- $\Delta t$  est le pas de temps de la simulation (s)

La force de poussée générée par les hélices est calculée par l'équation suivante :

$$F_p = 4 * KP * \left( \frac{2\pi}{60} \right)^2 * u^2 \quad (27)$$

Où :

- $KP$  est la constante de poussée d'une hélice
- $u$  est la vitesse de rotation des hélices (tour/min)

Parmi les constantes physiques du drone qui sont paramétrées comme ceci :

$$m = 0.2, \quad g = 9.81, \quad KP = 0.29, \quad C_d = 0.8 \quad (28)$$

Pour la configuration de l'anti-windup clamp, nous avons choisi de limiter le terme intégral à une valeur maximale de 30 et une valeur minimale de -30.

Pour la configuration de l'anti-windup back-calculation, nous avons choisi un gain  $K_{aw} = \frac{1}{0.2}$

### 5.2.5 Evaluation des performances

Pour évaluer les performances du système de contrôle avec et sans anti-windup, nous avons simulé le même scénario dans des conditions parfaites, c'est-à-dire sans la retenue de 10 secondes. Pour qu'on puisse se rendre compte des performances des anti-windups dans ces conditions.

Parmi les critères d'évaluation, nous avons choisi de mesurer l'altitude maximale atteinte par le drone, le dépassement maximal de l'altitude cible<sup>2</sup>, le temps de stabilisation du système, l'intégrale maximale de l'erreur et les commandes maximales, minimales et moyennes envoyées aux hélices.

Avec ces critères d'évaluation, nous avons pu comparer les performances du système de contrôle avec et sans anti-windup, et ainsi déterminer l'impact de l'anti-windup sur le comportement du drone.

Les critères d'évaluation les plus pertinents sont le dépassement maximal de l'altitude cible et la stabilisation du drone. On peut observer que l'anti-windup permet de réduire le dépassement maximal de l'altitude cible drastiquement. En effet, sans anti-windup le drone dépasse l'altitude cible de 140% ce qui va correspondre à une altitude de 12 mètres, alors qu'avec l'anti-windup clamp le dépassement est réduit à 46% ce qui va correspondre à une altitude de 7.32 mètres, et avec un anti-windup back-calculation ce dépassement va être réduit à 6.2% ce qui correspond à une altitude de 5.31 mètres (cf. tables 1b et 2b). Tout ceci est bien visible sur la figure 23.

Alors qu'avec le scénario dans les conditions parfaites, sans anti-windup le dépassement maximal de l'altitude cible est de 10% ce qui correspond à une altitude de 5.5 mètres, alors que l'anti-windup back-calculation permet de réduire ce dépassement à 6.2%, ce qui correspond à une altitude de 5.31 mètres (cf. tables 1a et 2a). Tout ceci est bien visible sur la figure 24.

Toutefois, il est important de noter que l'anti-windup a un impact sur le temps de stabilisation du système. En effet, dans le scénario avec la retenue de 10 secondes, le temps de stabilisation est de 32.2 secondes sans anti-windup, alors qu'avec l'anti-windup clamp il est réduit à 21.6 secondes et avec l'anti-windup back-calculation il est encore réduit à 18.9 secondes (cf. tables 3b). Ces temps de stabilisation sont assez longs, dus au fait que l'intégrale du PID atteint une valeur trop élevée, ce qui fait que le contrôleur met du temps à stabiliser la vitesse de rotation des hélices. Ces problèmes sont bien visibles sur la figure 27 où l'on peut voir que la commande du contrôleur est saturée à 15 tours/minute pendant un certain temps, puis passe à un minimum local de 7.13 tours/minute sans anti-windup, à 10.82 tours/minute avec l'anti-windup clamp et à 12.21 tours/minute avec l'anti-windup back-calculation. Ce qui fait qu'on a un effet de "rebond" sur la commande du contrôleur, ce qui fait que le drone met du temps à stabiliser sa vitesse de rotation des hélices. (cf. tables 5b et 6b)

---

<sup>2</sup>Signifie le pourcentage que représente l'écart entre l'altitude atteinte et l'altitude cible par rapport à l'altitude cible

| (a) Sans retenue de 10 secondes |                  | (b) Avec retenue de 10 secondes |                  |
|---------------------------------|------------------|---------------------------------|------------------|
|                                 | Altitude max (m) |                                 | Altitude max (m) |
| Sans anti-windup                | 5.5              | Sans anti-windup                | 12               |
| Anti-windup clamp               | 5.5              | Anti-windup clamp               | 7.32             |
| Anti-windup back-calculation    | 5.31             | Anti-windup back-calculation    | 5.31             |

Table 1: Comparaison des résultats avec et sans retenue sur l'altitude maximale atteinte par le drone.

Alors que dans le scénario sans la retenue de 10 secondes, le temps de stabilisation est de 11.8 secondes sans anti-windup, et avec l'anti-windup back-calculation il est réduit à 11.3 secondes (cf. tables 3a). Ces temps de stabilisation sont plus courts, dus au fait que l'intégrale du PID n'atteint pas une valeur trop élevée, ce qui fait que le contrôleur met moins de temps à stabiliser la vitesse de rotation des hélices. Ces problèmes sont bien visibles sur la figure 28 où l'on peut voir que la commande du contrôleur est saturée à 15 tours/minute pendant un certain temps, puis passe à un minimum local de 12.08 tours/minute sans anti-windup, à 12.21 tours/minute avec l'anti-windup back-calculation. Ce qui fait qu'on a un effet de "rebond" sur la commande du contrôleur, mais moins prononcé que dans le scénario avec la retenue de 10 secondes. (cf. tables 5a et 6a)

#### Remarque

On ne fait pas mention de l'anti-windup clamp dans le scénario sans la retenue de 10 secondes, car il n'y a pas d'activation par rapport à l'intégrale du PID, donc il n'y a pas de différence avec le PID sans anti-windup.

On peut donc conclure que l'utilisation d'un anti-windup est bénéfique dans ces scénarios, car il permet de réduire le dépassement maximal de l'altitude cible et le temps de stabilisation du système, ce qui est crucial pour le contrôle d'un drone. Cependant, il est important de noter que l'anti-windup a un impact sur la commande du contrôleur, et qu'il est donc nécessaire de bien choisir les paramètres de l'anti-windup en fonction du système à contrôler.

L'anti-windup back-calculation est plus efficace que l'anti-windup clamp, car il permet de réduire le dépassement maximal de l'altitude cible et le temps de stabilisation du système, tout en limitant l'impact sur la commande du contrôleur. Cependant, il est plus complexe à mettre en place que l'anti-windup clamp, et il est donc nécessaire de bien comprendre son fonctionnement avant de l'implémenter.

Dans des conditions parfaites, on peut avoir l'impression que l'anti-windup n'est pas nécessaire, car le dépassement maximal de l'altitude cible est déjà faible, et le temps de stabilisation du système est déjà court. Cependant, dans des conditions réelles, telles que dans un intérieur de bâtiment, on peut vouloir que le drone n'atteigne jamais 5.5 mètres car il pourrait heurter le plafond, et donc l'anti-windup est nécessaire pour éviter ce genre de situation.

| (a) Sans retenue de 10 secondes |                         | (b) Avec retenue de 10 secondes |                         |
|---------------------------------|-------------------------|---------------------------------|-------------------------|
|                                 | Dépassement maximal (%) |                                 | Dépassement maximal (%) |
| Sans anti-windup                | 10                      | Sans anti-windup                | 140                     |
| Anti-windup clamp               | 10                      | Anti-windup clamp               | 46                      |
| Anti-windup back-calculation    | 6.2                     | Anti-windup back-calculation    | 6.2                     |

Table 2: Comparaison des résultats avec et sans retenue sur le dépassement maximal de l'altitude atteinte par le drone.

| (a) Sans retenue de 10 secondes |                                          | (b) Avec retenue de 10 secondes |                                          |
|---------------------------------|------------------------------------------|---------------------------------|------------------------------------------|
|                                 | Temps de stabilisation ( $\pm 5\%$ ) (s) |                                 | Temps de stabilisation ( $\pm 5\%$ ) (s) |
| Sans anti-windup                | 11.8                                     | Sans anti-windup                | 32.2                                     |
| Anti-windup clamp               | 11.8                                     | Anti-windup clamp               | 21.6                                     |
| Anti-windup back-calculation    | 11.3                                     | Anti-windup back-calculation    | 18.9                                     |

Table 3: Comparaison des résultats avec et sans retenue sur le temps de stabilisation du drone.

| (a) Sans retenue de 10 secondes |                    | (b) Avec retenue de 10 secondes |                    |
|---------------------------------|--------------------|---------------------------------|--------------------|
|                                 | Intégrale maximale |                                 | Intégrale maximale |
| Sans anti-windup                | 22.6               | Sans anti-windup                | 60.86              |
| Anti-windup clamp               | 22.6               | Anti-windup clamp               | 30                 |
| Anti-windup back-calculation    | 21.9               | Anti-windup back-calculation    | 21.9               |

Table 4: Comparaison des résultats avec et sans retenue sur l'intégrale maximale atteinte par le contrôleur du drone.

| (a) Sans retenue de 10 secondes |                              | (b) Avec retenue de 10 secondes |                              |
|---------------------------------|------------------------------|---------------------------------|------------------------------|
|                                 | Commande maximale (tour/min) |                                 | Commande maximale (tour/min) |
| Sans anti-windup                | 15                           | Sans anti-windup                | 15                           |
| Anti-windup clamp               | 15                           | Anti-windup clamp               | 15                           |
| Anti-windup back-calculation    | 15                           | Anti-windup back-calculation    | 15                           |

Table 5: Comparaison des résultats avec et sans retenue sur la commande maximale générée par le contrôleur du drone.



| (a) Sans retenue de 10 secondes |                                    | (b) Avec retenue de 10 secondes |                                    |
|---------------------------------|------------------------------------|---------------------------------|------------------------------------|
|                                 | Commande minimale local (tour/min) |                                 | Commande minimale local (tour/min) |
| Sans anti-windup                | 12.08                              | Sans anti-windup                | 7.13                               |
| Anti-windup clamp               | 12.08                              | Anti-windup clamp               | 10.82                              |
| Anti-windup back-calculation    | 12.21                              | Anti-windup back-calculation    | 12.21                              |

Table 6: Comparaison des résultats avec et sans retenue sur la commande minimale locale générée par le contrôleur du drone.

| (a) Sans retenue de 10 secondes |                             | (b) Avec retenue de 10 secondes |                             |
|---------------------------------|-----------------------------|---------------------------------|-----------------------------|
|                                 | Commande moyenne (tour/min) |                                 | Commande moyenne (tour/min) |
| Sans anti-windup                | 12.54                       | Sans anti-windup                | 12.78                       |
| Anti-windup clamp               | 12.54                       | Anti-windup clamp               | 12.86                       |
| Anti-windup back-calculation    | 12.54                       | Anti-windup back-calculation    | 12.88                       |

Table 7: Comparaison des résultats avec et sans retenue sur la commande moyenne générée par le contrôleur du drone.

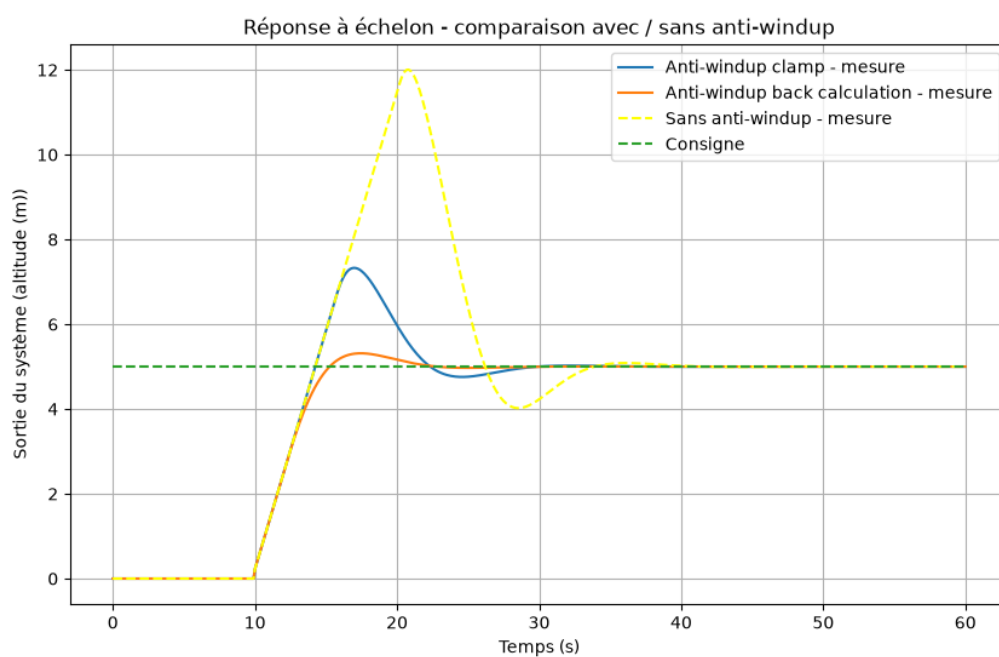


Figure 23: Altitude du drone en avec différents anti-windups avec la retenue

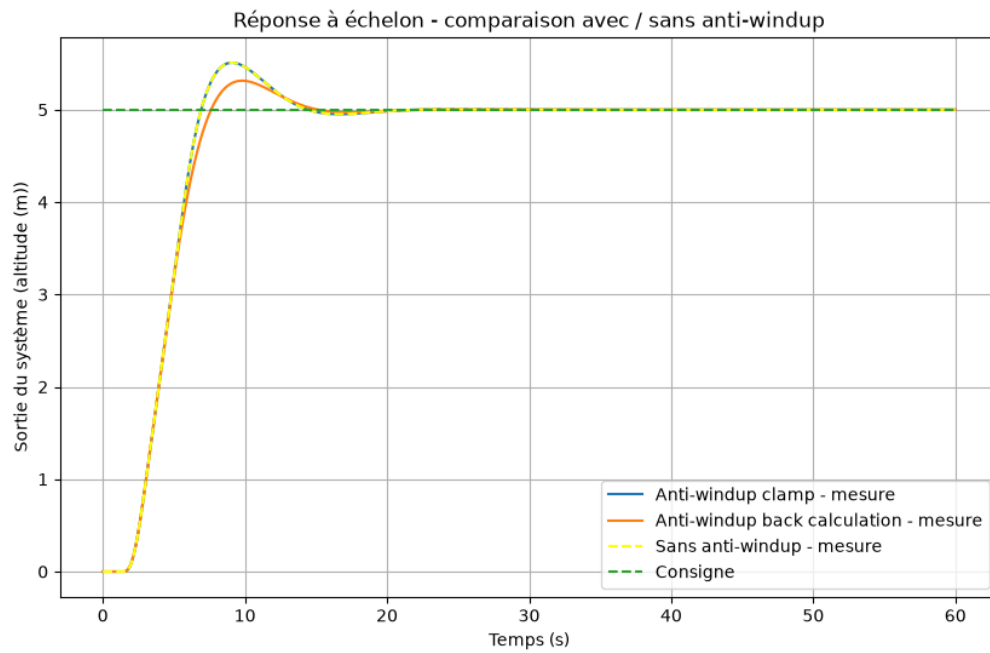


Figure 24: Altitude du drone en avec différents anti-windups sans la retenue

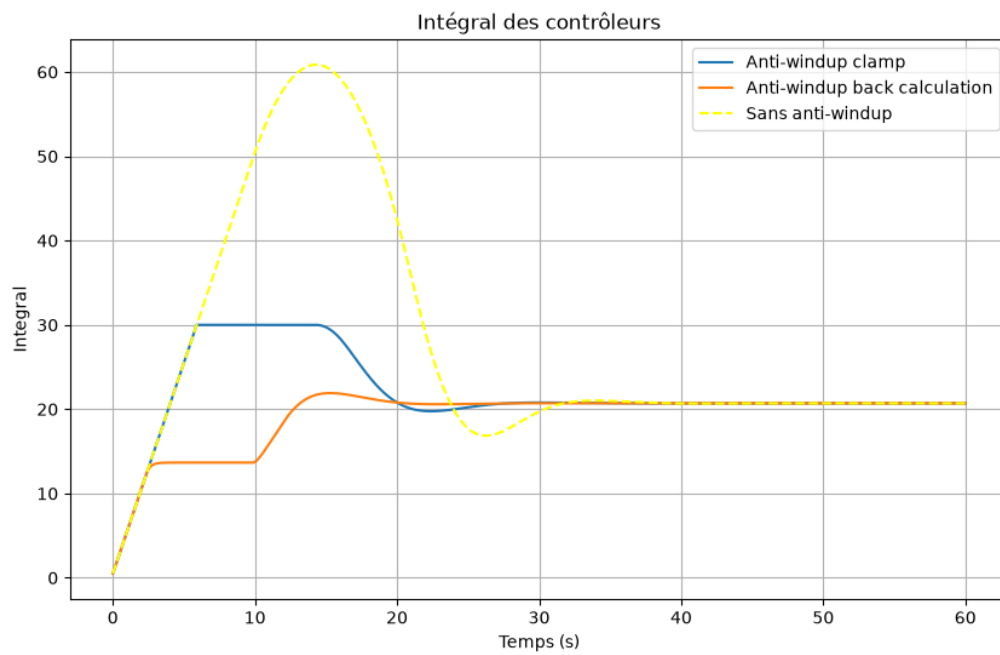


Figure 25: Intégrale des différents anti-windups avec la retenue

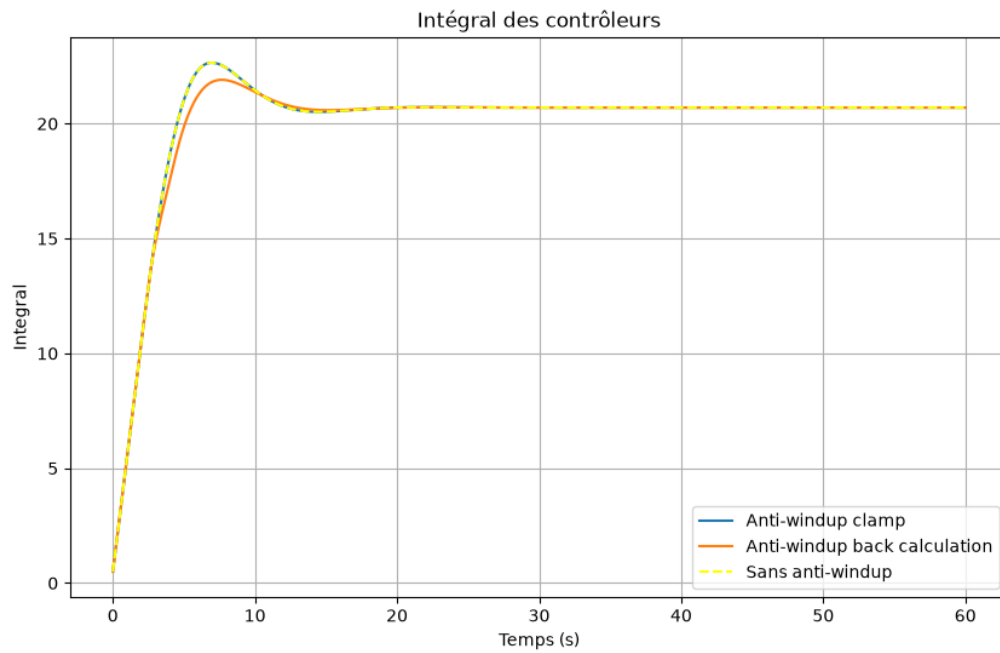


Figure 26: Intégrale des différents anti-windups sans la retenue

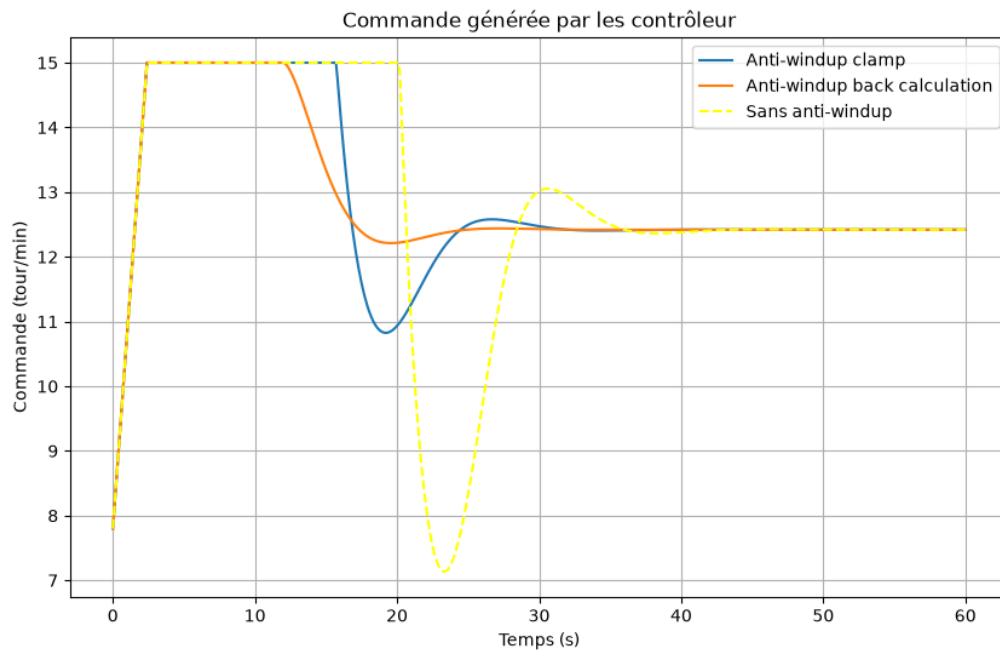


Figure 27: Vitesse de rotation des hélices avec différents anti-windups avec retenue

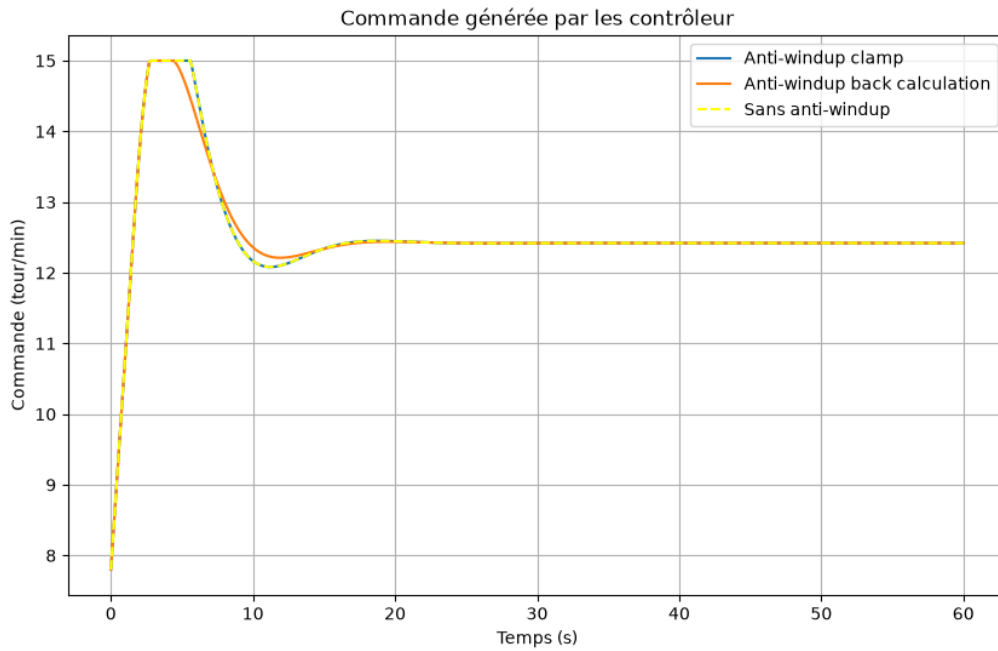


Figure 28: Vitesse de rotation des hélices avec différents anti-windups sans retenue

### 5.3 La chaîne d'outils

Maintenant que nous avons vu les différents outils de la théorie du contrôle que nous avons implémentés pour CHIPS, nous allons voir comment nous avons mis en place la chaîne d'outils permettant d'à partir d'un programme CHIPS, de générer un programme BIP, qui sera ensuite compilé et exécuté pour obtenir les résultats de simulation.

#### 5.3.1 Le fonctionnement de la chaîne d'outils

La chaîne d'outils fonctionne de la manière suivante : à partir d'un programme CHIPS, le parseur développé lors de la partie 4.1 va générer un AST. Celui-ci sera ensuite utilisé par le programme qui va le sérialiser en un fichier XMI. Ce fichier sera ensuite analysé par un programme python pour vérifier sa validité par rapport au méta-modèle. Si le fichier est valide, alors une autre partie du compilateur y applique des règles de transformation ATL pour générer un fichier XMI, qui lui sera valide par rapport au méta-modèle BIP. Enfin, ce dernier fichier XMI sera utilisé par le compilateur BIP pour générer un programme BIP exécutable. Pour l'instant ce programme ne contient que les composants ainsi que les connecteurs, mais pour l'instant il n'y a pas la logique interne des composants.

Une fois le programme BIP généré, on peut écrire la logique interne des composants en BIP, puis générer du code C++ à nouveau en passant par le compilateur BIP. Ce code C++ peut ensuite être compilé et exécuté pour obtenir les résultats de simulation.

#### 5.3.2 Les résultats de la chaîne d'outils

Pour illustrer le fonctionnement de la chaîne d'outils, nous avons réutilisé l'exemple du drone que nous avons utilisé dans la partie 5.2. Nous avons donc écrit un programme CHIPS pour le

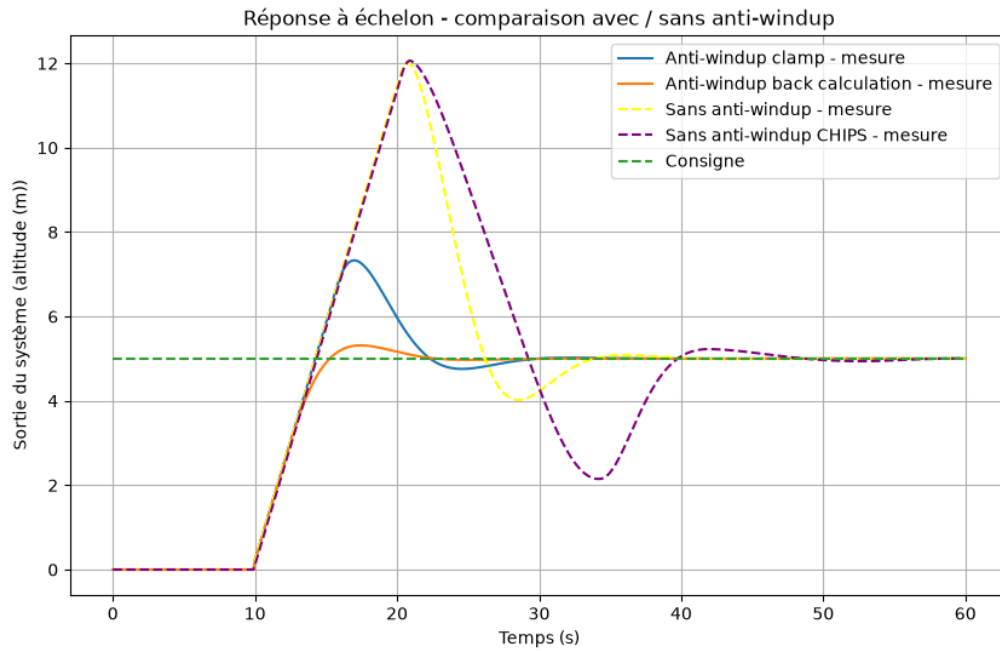


Figure 29: Les différentes altitudes du drone avec différents anti-windups comparés à sans anti-windup en CHIPS

drone, puis nous avons utilisé la chaîne d'outils pour générer un programme BIP, puis du code C++ et enfin exécuter le programme pour obtenir les résultats de simulation.

Nous avons trois programmes BIP différents<sup>3</sup>, un sans anti-windup, un avec l'anti-windup clamp et un avec l'anti-windup back-calculation. Nous avons ensuite exécuté les trois programmes pour obtenir les résultats de simulation. Les résultats sont présentés dans les figures 29, 30, 31, 32, 33, 34, 35, 36 et 37.

Une différence flagrante est à remarquer, les résultats obtenus avec la chaîne d'outils ne sont pas les mêmes que ceux obtenus lorsqu'on exécute un programme C++ directement. Cette différence s'explique par le fait que la logique interne du composant PID en BIP utilise des fonctions C++ qui utilisent toutes en type pour ses variables des float, alors qu'en BIP, les variables sont de type double. Cette différence de précision peut donc expliquer les différences de résultats.

Néanmoins, même si les résultats ne sont pas les mêmes, ils suivent la même tendance et permettent de comparer les différents anti-windups entre eux. On peut donc conclure que la chaîne d'outils fonctionne correctement et permet de générer un programme BIP à partir d'un programme CHIPS.

<sup>3</sup>Etant donné qu'il n'y a pas la logique interne des composants en BIP les programme CHIPS se ressemblent dû à leurs composantes et leurs connections

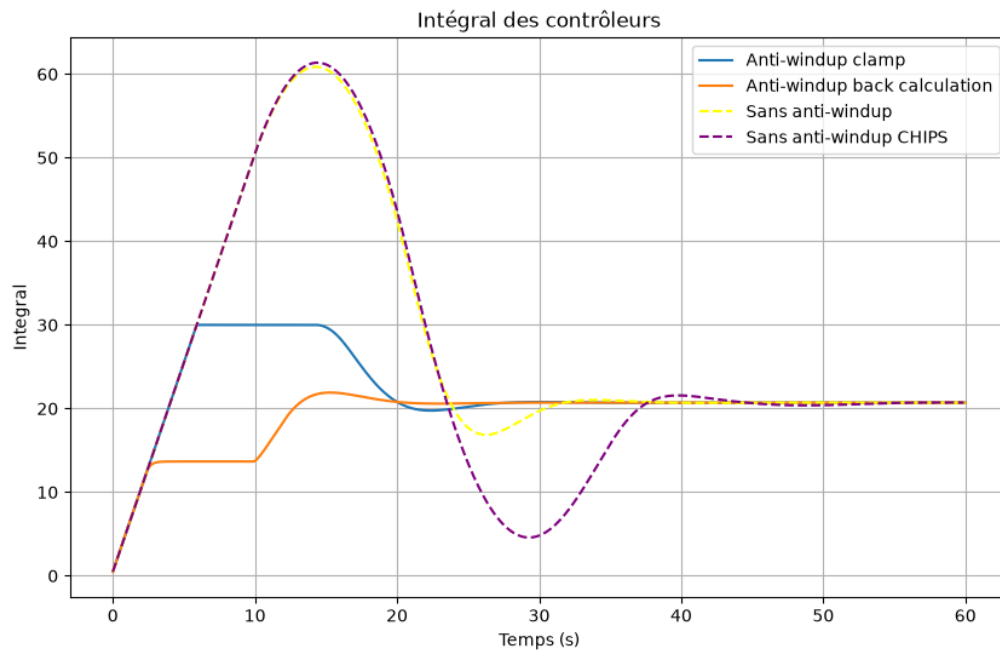


Figure 30: Les différentes intégrales des anti-windups comparées à sans anti-windup en CHIPS

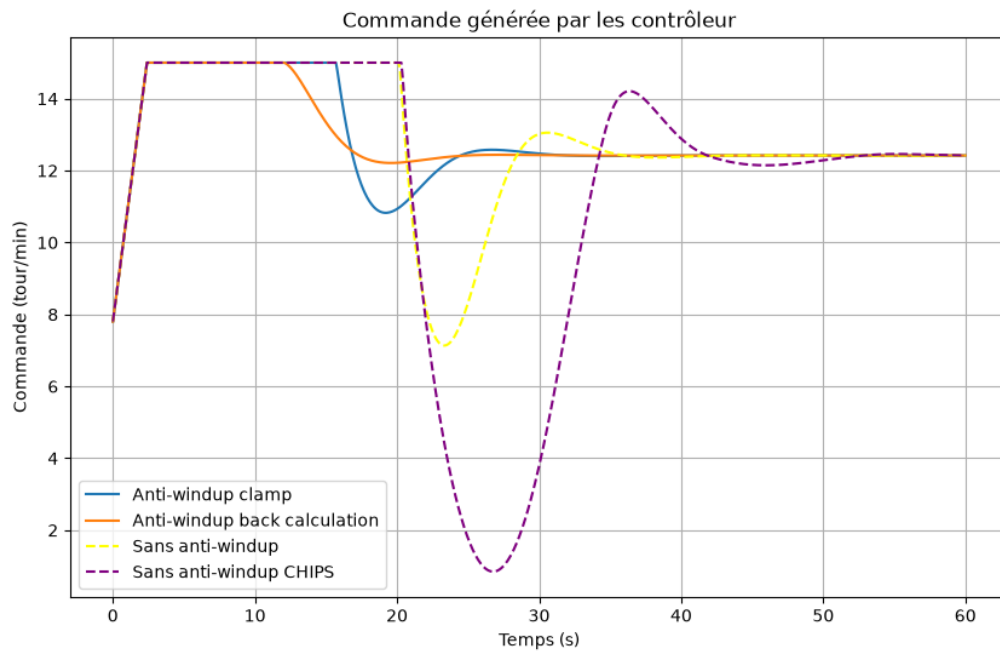


Figure 31: Les différentes vitesses de rotation des hélices avec différents anti-windups comparées à sans anti-windup en CHIPS

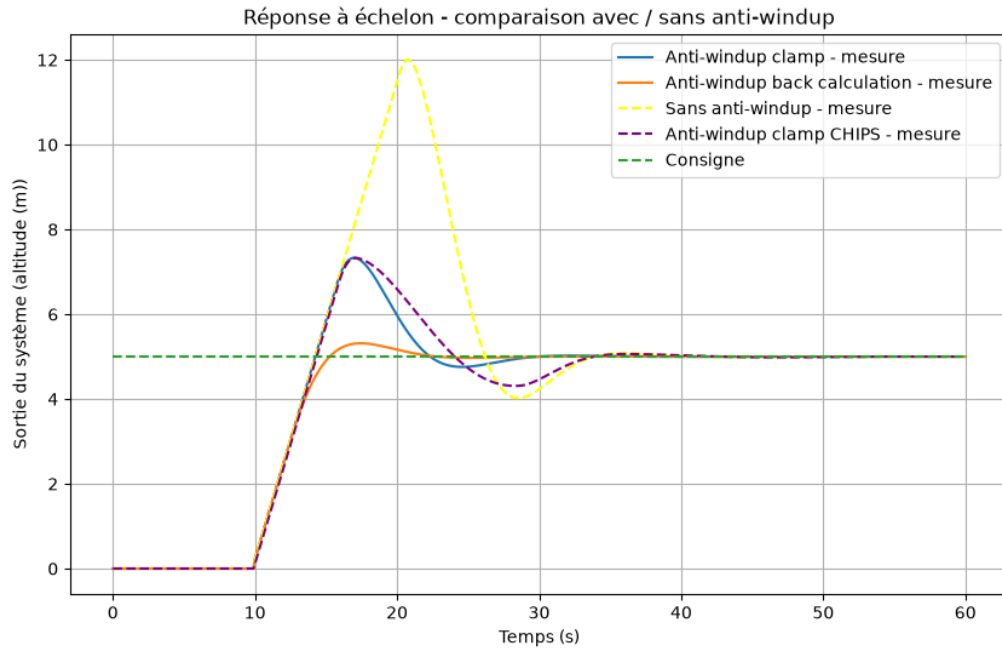


Figure 32: Les différentes altitudes du drone avec différents anti-windups comparés à l'anti-windup clamp en CHIPS

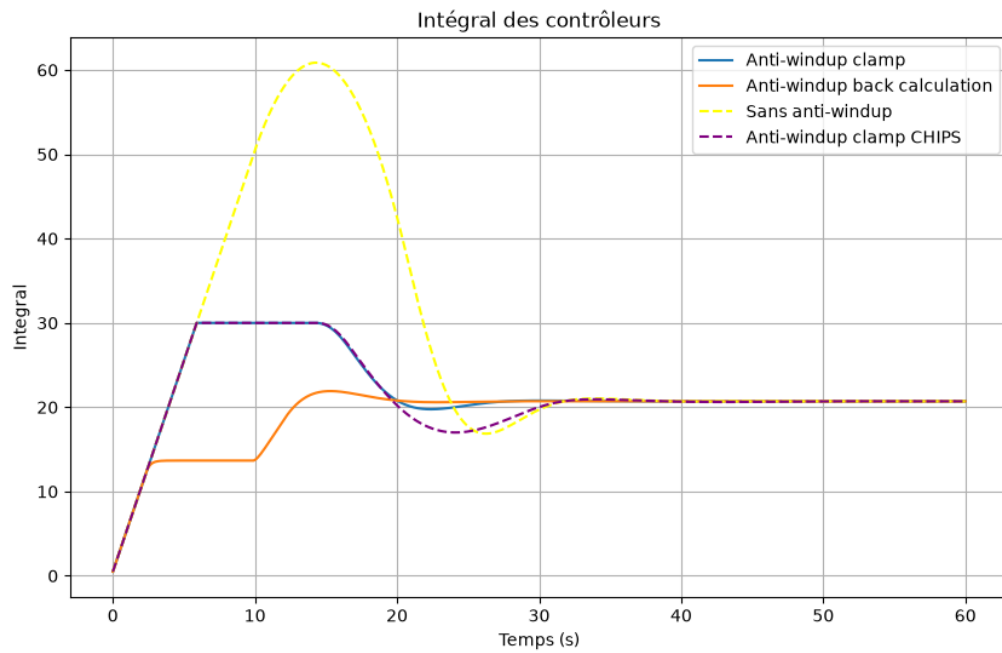


Figure 33: Les différentes intégrales des anti-windups comparées à l'anti-windup clamp en CHIPS

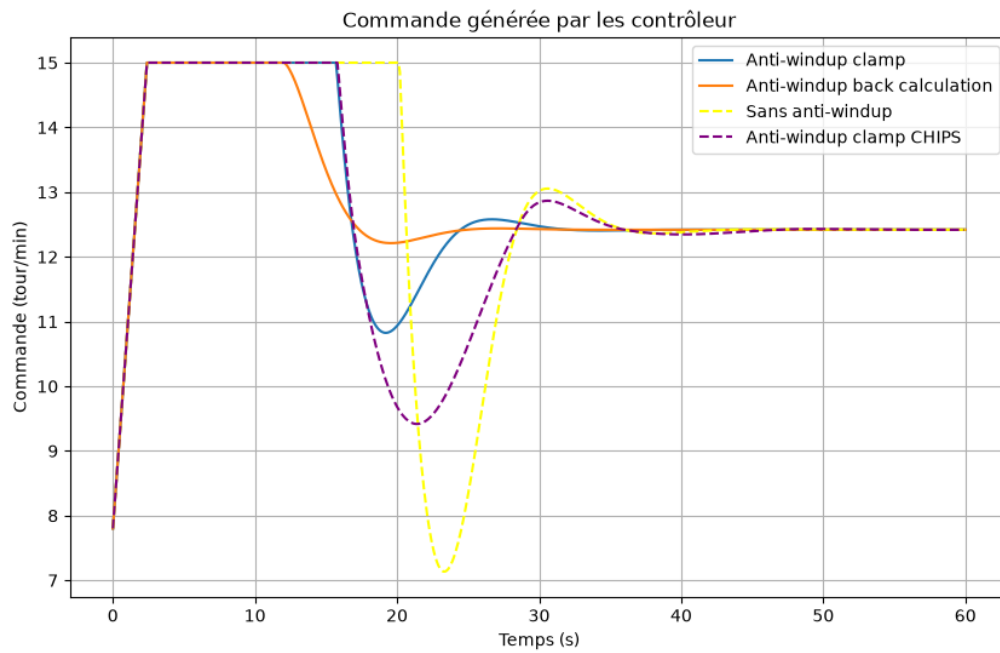


Figure 34: Les différentes vitesses de rotation des hélices avec différents anti-windups comparés à l'anti-windup clamp en CHIPS

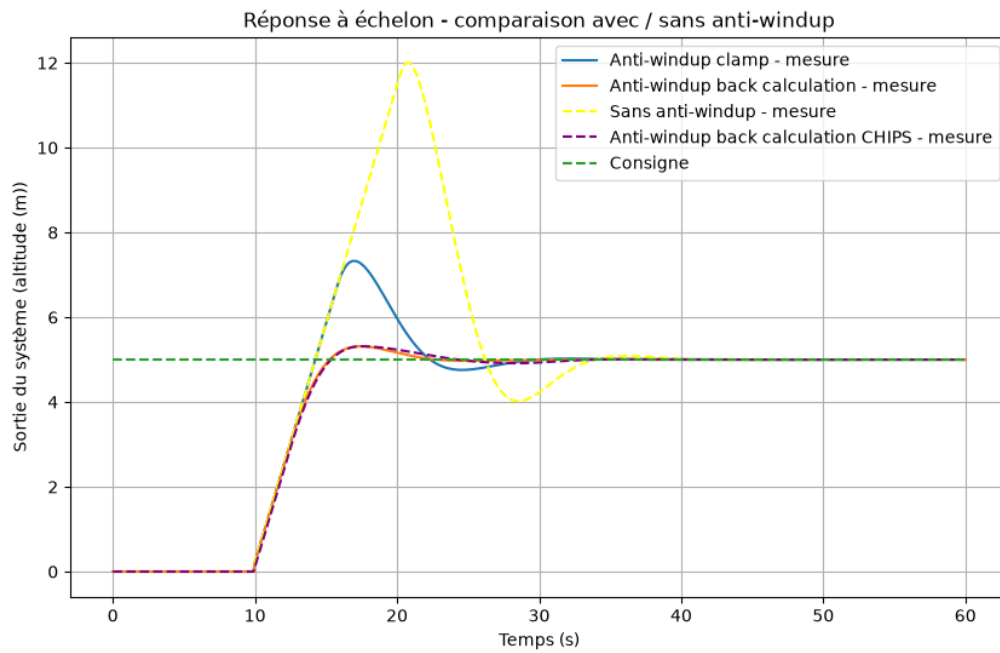


Figure 35: Les différentes altitudes du drone avec différents anti-windups comparés à l'anti-windup back-calculation en CHIPS



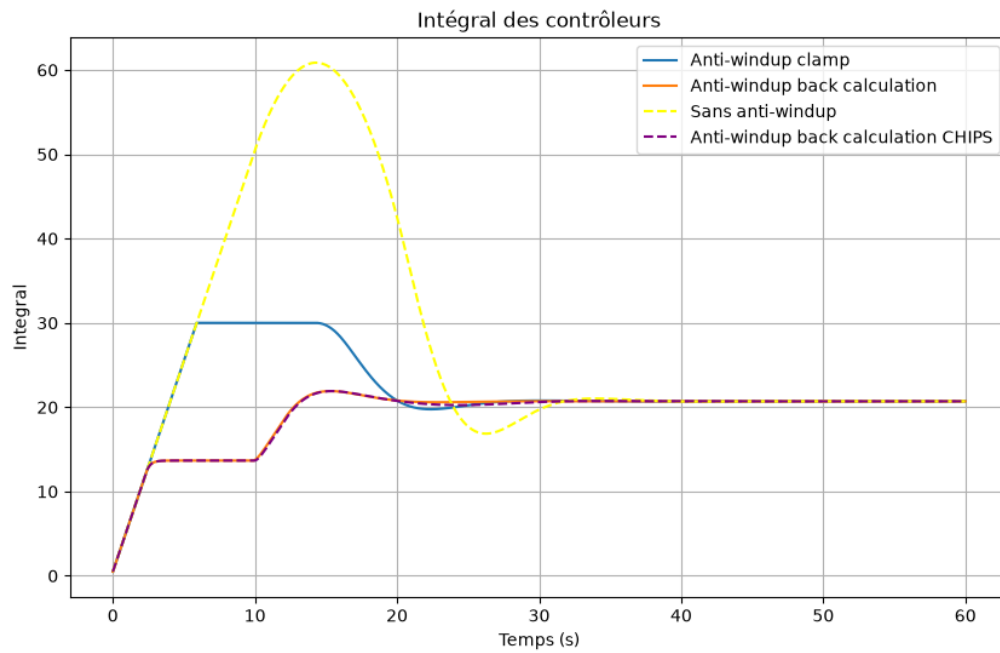


Figure 36: Les différentes intégrales des anti-windups comparées à l'anti-windup back-calculation en CHIPS

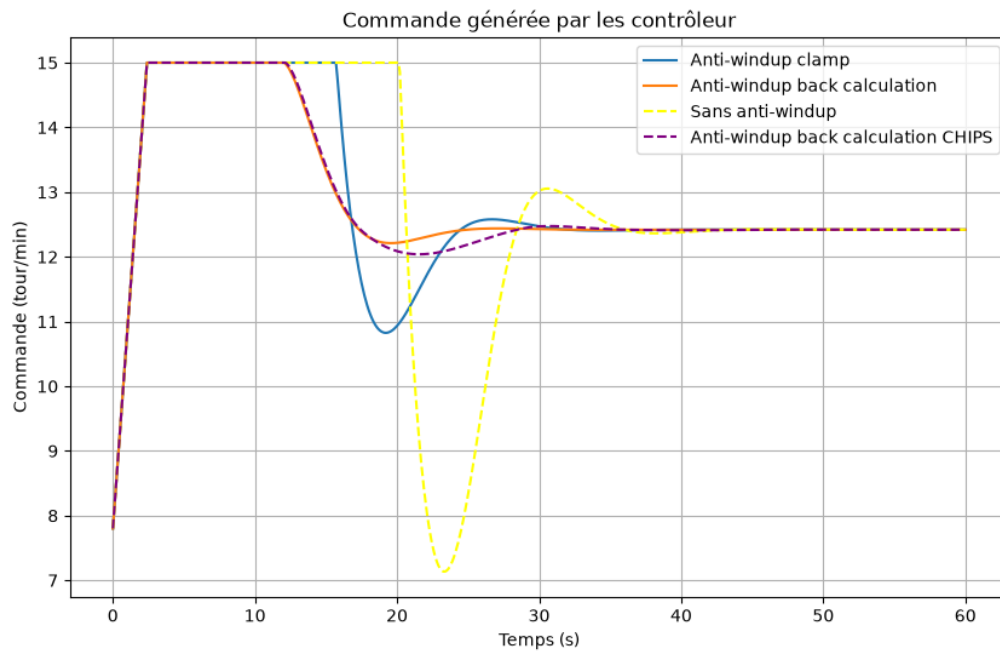


Figure 37: Les différentes vitesses de rotation des hélices avec différents anti-windups comparés à l'anti-windup back-calculation en CHIPS

### 5.3.3 Les métriques de la chaîne d'outils

Grâce à la chaîne d'outils, à partir d'un programme CHIPS qui fait 65 lignes, on obtient un programme BIP qui fait 600 lignes de code avec toute la logique interne des composants. Ce qui montre qu'un programme CHIPS est beaucoup plus compact qu'un programme BIP et qu'il abstrait certains détails d'implémentation de BIP.

Nous avons mesuré les différents temps d'exécution de chacun des processus de la chaîne de compilation. Ces mesures ont été effectuées à partir du programme CHIPS modélisant le drone. Pour la partie de l'interprétation du programme CHIPS à la génération du programme BIP, il faut 10.229702854 secondes, pour la génération du code C++ à partir du programme BIP ainsi que sa compilation il faut 227.576063377 secondes, ce qui fait un temps total de 237.805766231 secondes, ce qui correspond à 3 minutes et 57 secondes pour la chaîne d'outils complète.

Pour information, la chaîne d'outils a été exécutée sur un ordinateur avec les caractéristiques suivantes :

- Système d'exploitation : Ubuntu 24.04.4 LTS
- Processeur : Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz
- Mémoire vive : 16 GB DDR4

## 6 Conclusion

Ce stage a eu pour objectif de finir le développement du parseur et la sérialisation vers le format XMI, d'implémenter des outils de la théorie du contrôle, des filtres et des anti-windups pour CHIPS, ainsi que d'unifier tout cela pour avoir une chaîne d'outils complète et fonctionnelle.

### 6.1 Bilan technique

D'un point de vue technique, le parseur est maintenant fonctionnel et on peut générer des fichiers XMI, conforme au méta-modèle, à partir du programme CHIPS.

Au total, 19 classes C++ en rapport avec la théorie du contrôle ont été implémentées, que ce soit des filtres ou des anti-windups. 10 bancs de tests ont été réalisés, avec pour chacun d'eux des graphiques et des résultats de simulation. L'ensemble de ces outils peuvent être utilisés avec CHIPS car nous avons mis en place une chaîne d'outils complète et fonctionnelle, qui permet de générer un programme BIP à partir d'un programme CHIPS, puis de générer du code C++ à partir de ce programme BIP, et enfin d'exécuter le programme pour obtenir les résultats de simulation.

Tout cela correspond à un volume de d'un peu plus de 10000 lignes de code C++, python et script bash.

### 6.2 Les perspectives

Différentes perspectives d'amélioration sont possibles pour ce projet. Tout d'abord, il serait intéressant de mettre en place des tests de non-régression pour que les modifications futures du parseur ne cassent pas les fonctionnalités existantes.

Ensuite pour la partie des filtres et des anti-windups, il faudrait utiliser le même type de données pour les variables pour éviter les différences de résultats engendrés par les différences de précision. Sur la même catégorie, il faudrait également implémenter des tests unitaires pour les classes de la théorie du contrôle pour s'assurer de leur bon fonctionnement.

Aussi, de pouvoir utiliser les filtres et les anti-windups directement dans un programme CHIPS, tout cela a déjà été réfléchi mais par manque de temps, cela n'a pas pu être implémenté.

Enfin, il serait intéressant de pouvoir avoir d'autres outils de la théorie du contrôle, comme le gain scheduling, le contrôle par avance/retard de phase...

### **6.3 Bilan personnel**

D'un point de vue personnel, ce stage s'est inscrit dans la suite logique de mon projet semestriel, débuté depuis octobre 2025, qui avait été l'implémentation du parseur et la sérialisation vers XMI avec deux autres personnes. Cela a été pour moi le plus long projet sur lequel j'ai pu travailler car il a duré au total 9 mois.

Ce stage m'a permis de développer mes compétences techniques en C++, ainsi que sur la modélisation d'un langage de programmation et de comprendre tous les tenants et aboutissant.

Cette expérience m'a permis de me rendre compte de l'importance de la communication lors des comptes rendus d'avancement, car cela permet de faire le point sur l'avancement du projet et de discuter des problèmes rencontrés.

Cela m'a également permis de mettre en pratique ainsi qu'approfondir mes connaissances sur le cours de Systèmes Cyber-Physiques (M2).

## A Annexe

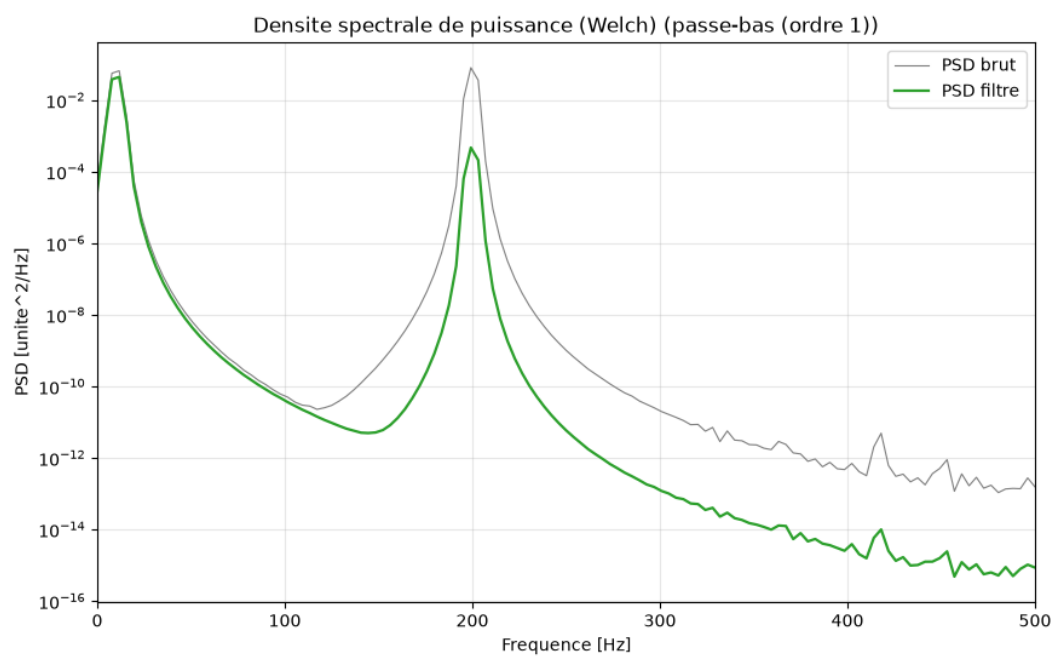


Figure 38: Densité spectrale de puissance avant et après filtre passe-bas d'ordre 1

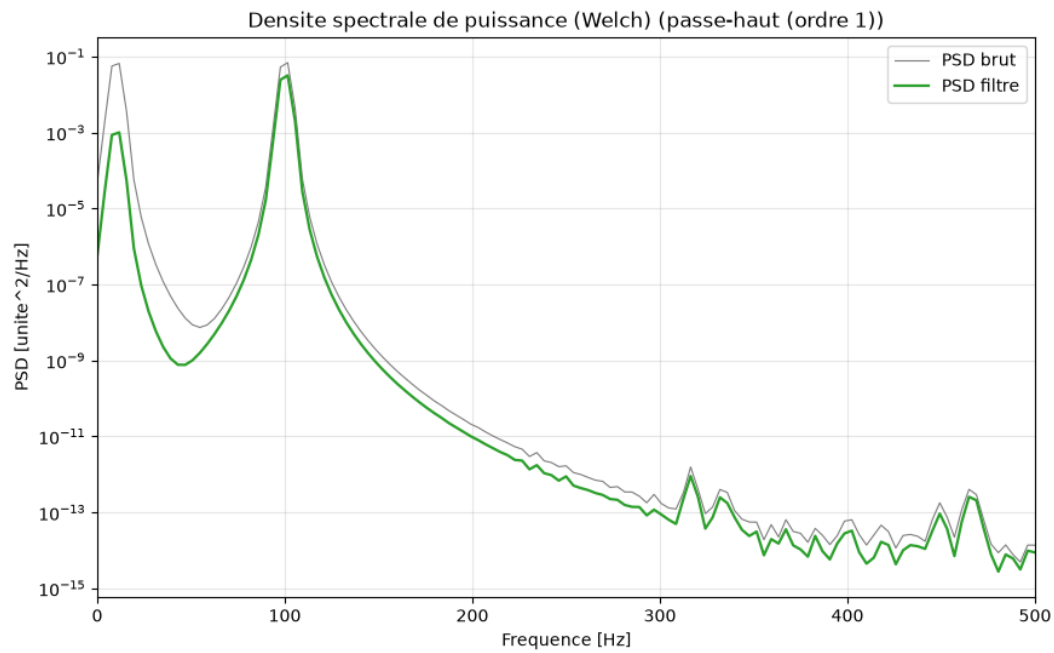


Figure 39: Densité spectrale de puissance avant et après filtre passe-haut d'ordre 1

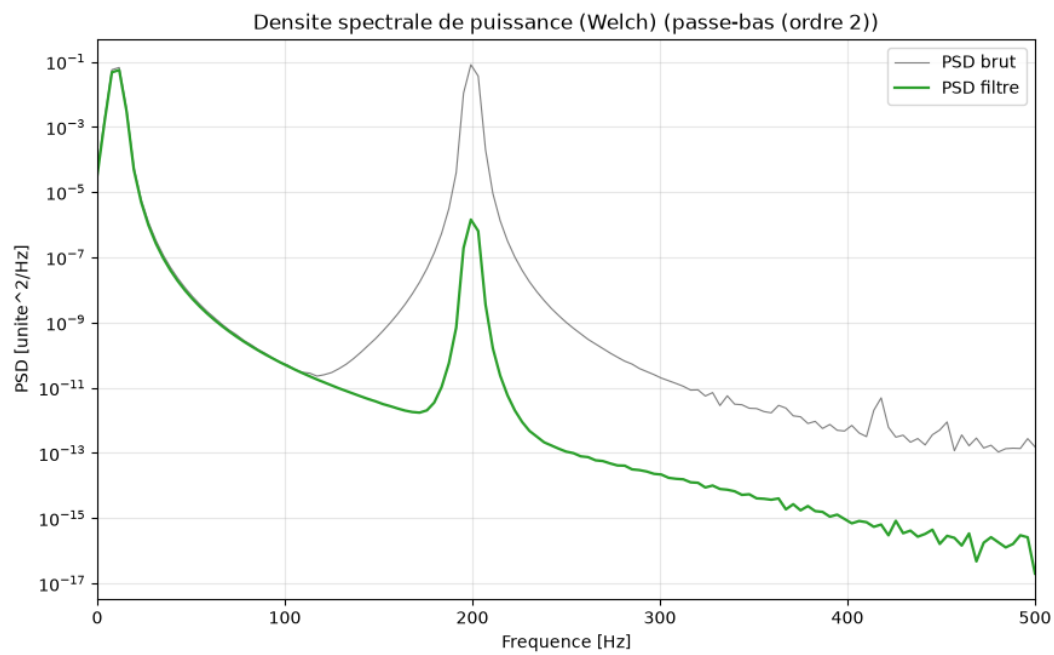


Figure 40: Densité spectrale de puissance avant et après filtre passe-bas d'ordre 2

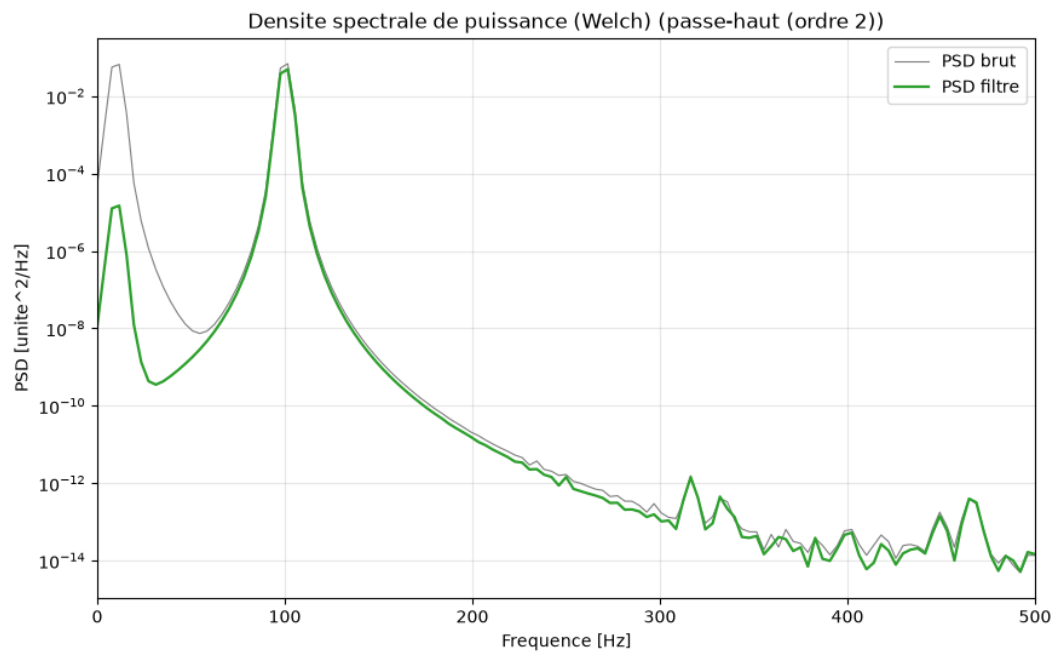


Figure 41: Densité spectrale de puissance avant et après filtre passe-haut d'ordre 2

```

1 logical Drone(float u, float dt) init {
2     float m = 0.2;
3     float altitude = 0.;
4     float altitude_max = 0.;
5     float velocity = 0.;
6     float prevError = 0.;
7     float GRAVITY = 9.81;
8     float KP = 0.29;
9     float DRAG_COEFF = 0.8;
10 }then{
11     float radian = ((2. * 3.14159265358979323846) / 60.);
12     float K = 4. * KP * radian * radian;
13     float Fp = K * u * u;
14     float drag = DRAG_COEFF * velocity;
15     float a = (Fp / m) - GRAVITY - (drag / m);
16     velocity = velocity + a * dt;
17     altitude = altitude + velocity * dt;
18     if(altitude < 0.){
19         altitude = 0.;
20         velocity = 0.;
21     }
22     if(altitude_max < altitude){
23         altitude_max = altitude;
24     }else{
25         altitude_max = altitude_max;
26     }
27 } -> update(altitude)
28
29 logical PID(float target, float current, float dt) init {
30     float prevError = 0.;
31     float proportional = 0.;
32     float integral = 0.;
33     float derivative = 0.;
34     float kp = 1.5;
35     float ki = 0.6;
36     float kd = 0.0;
37     float minOutput = 0.0;
38     float maxOutput = 15.0;
39 }then{
40     float error = target - current;
41     float p_out = kp * error;
42     proportional = error;
43     integral = integral + error * dt;
44     float i_out = ki * integral;
45     derivative = 0.;
46     if(dt > 0.){
47         derivative = (error - prevError) / dt;
48     }
49     float d_out = kd * derivative;
50     float rawCorrection = p_out + i_out + d_out;
51     float saturated = (float)(max((int)(minOutput), min((int)(rawCorrection), (int)(
        maxOutput)))));
52     prevError = error;
53 } -> correction(saturated)
54
55 system {
56     PID ctrl;

```



```
57   Drone drone;
58   float dt = 0.1;
59   float target = 5.0;
60   ctrl.dt(dt);
61   ctrl.target(target);
62   ctrl.current(drone.update);
63   drone.u(ctrl.correction);
64   drone.dt(dt);
65 }
```

Listing 2: Exemple de programme CHIPS pour la simulation d'un drone avec un contrôleur PID

## B Glossaire

- CHIPS : Control of Hierarchical Interconnected Programmable Systems
- BIP : Behavior, Interaction, Priority
- PID : Proportional-Integral-Derivative
- ANTLR4 : ANother Tool for Language Recognition version 4
- XMI : XML Metadata Interchange
- ATL : ATLAS Transformation Language
- AST : Abstract Syntax Tree
- PSD : Power Spectral Density
- FFT : Fast Fourier Transform

## Références

- [1] Karl Johan ÅSTRÖM et Richard M. MURRAY. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.
- [2] Ananda BASU, Marius BOZGA et Joseph SIFAKIS. “Modeling Heterogeneous Real-time Components in BIP”. In : *Software Tools for Technology Transfer* 12.2 (2011), p. 91-118.
- [3] Stephen BUTTERWORTH. “On the Theory of Filter Amplifiers”. In : *Wireless Engineer* 7 (1930), p. 536-541.
- [4] Alan V. OPPENHEIM et Ronald W. SCHAFER. *Discrete-Time Signal Processing*. 3<sup>e</sup> éd. Prentice Hall, 2010.
- [5] Sophocles J. ORFANIDIS. *Introduction to Signal Processing*. Prentice Hall, 1996.
- [6] Sophie TARBOURIECH et Matthew TURNER. “Anti-windup design: An overview of some recent advances and open problems”. In : *IET Control Theory & Applications* 3.1 (2009), p. 1-19.

---

## Résumé

Ce rapport présente les travaux de développement et d'amélioration du compilateur pour le langage **CHIPS** (Control of Hierarchical Interconnected Programmable Systems), un langage dédié à la modélisation formelle de systèmes distribués complexes. Les objectifs principaux ont été de migrer le parseur historique vers ANTLR4 afin d'obtenir une architecture plus robuste et plus facilement extensible, d'adapter le générateur XMI au nouvel arbre syntaxique produit par ANTLR4, de développer une bibliothèque de filtres numériques comprenant des filtres passe-bas, passe-haut, Butterworth d'ordre 2 ainsi qu'un filtre médian, et d'implémenter plusieurs mécanismes d'anti-windup destinés aux contrôleurs PID, ainsi que d'évaluer expérimentalement les performances des différentes solutions proposées au moyen d'analyses temporelles, fréquentielles. Aussi de réaliser une chaîne de compilation complète pour le langage CHIPS intégrant l'ensemble de ces composants.

### Mots clés :

CHIPS, C++, ANTLR4, XMI, méta-modèle, filtres, anti-windup, PID

## Abstract

This report presents the development and enhancement of the compiler for the CHIPS (Control of Hierarchical Interconnected Programmable Systems) language, a domain-specific language dedicated to the formal modeling of complex distributed systems. The main objectives were to migrate the legacy parser to ANTLR4 in order to obtain a more robust and easily extensible architecture, to adapt the XMI generator to the new abstract syntax tree produced by ANTLR4, to develop a library of digital filters including first-order low-pass and high-pass filters, a second-order Butterworth filter, and a median filter, and to implement several anti-windup mechanisms for PID controllers. In addition, the proposed solutions were experimentally evaluated through both time-domain and frequency-domain analyses. Finally, a complete compilation toolchain for the CHIPS language was developed, integrating all of these components.

### Keywords :

CHIPS, C++, ANTLR4, XMI, meta-model, filters, anti-windup, PID

---